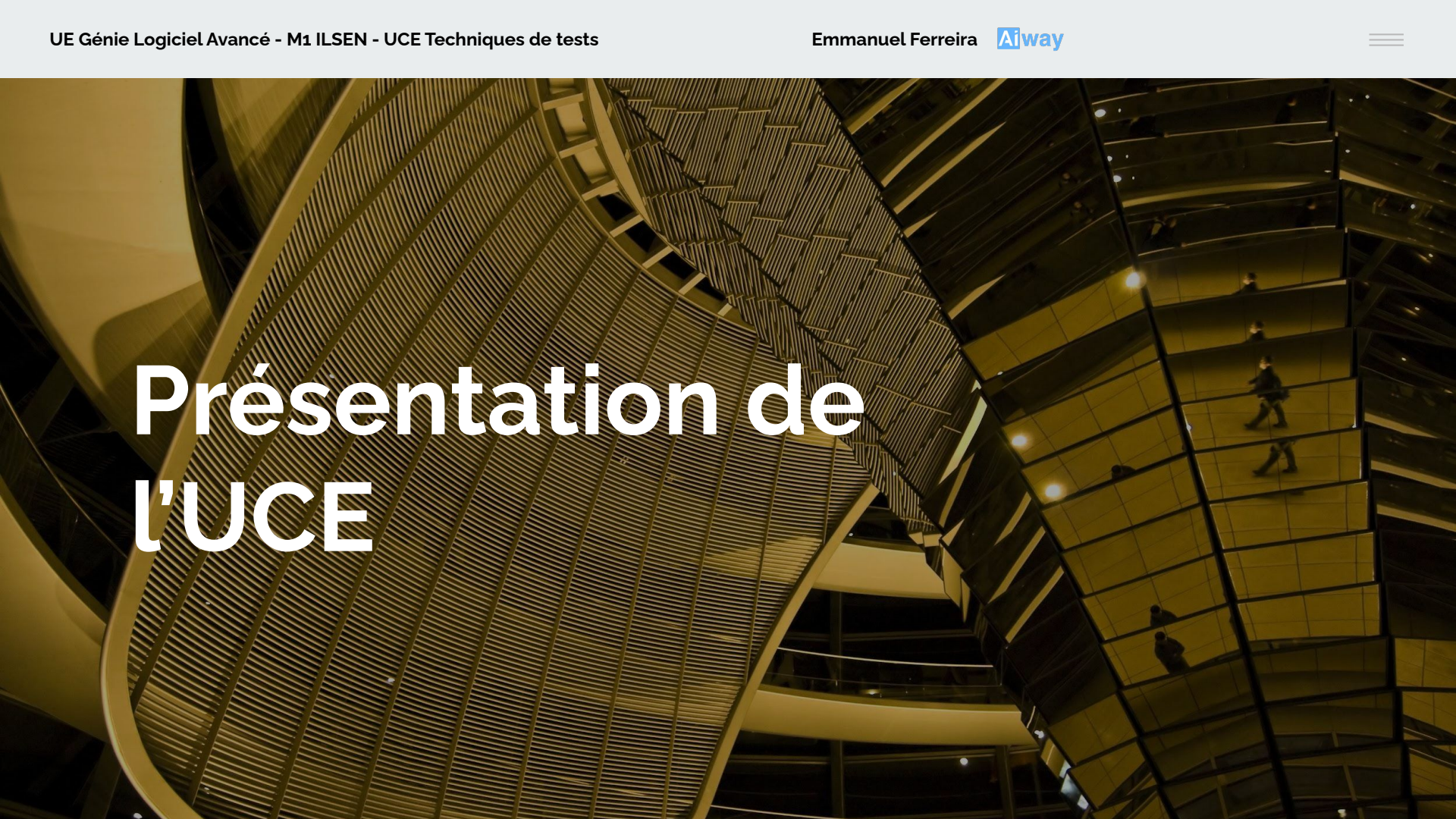




UCE Techniques de tests

CM 1 : Introduction des principaux concepts





Présentation de l'UCE



Les intervenants

- 2 Cours Magistraux
 - Emmanuel FERREIRA
 - Daniel SALAS
- TPs - 3 groupes
 - Ludovic BONNEFOY
 - Daniel SALAS
 - Ricardo RODRIGUEZ

Organisation de l'UCE

- Volume horaire UCE-FC
 - Cours : 6 heures
 - TP : 19 heures
 - Évaluation : 1,5 heures





Évaluation

- Une progression de 6 TPs (50%)
 - Rendus individuels
 - Deux jalons :
 - Rendu de mi-parcours (TPs 1 à 3) $\Rightarrow$ 15%
 - Rendu final + “mini-soutenance” (TPs 4 à 6) $\Rightarrow$ 35%
- Une évaluation écrite (50%)



Thématiques abordées dans ce cours

- Outils et techniques pour assurer diverses activités de vérification et de validation au cours d'un développement logiciel
 - Présentation des principaux types de tests & de la terminologie
 - Tests unitaires + Mocks
 - Qualimétrie
 - Tests de performance
 - Tests de sécurité
 - Tests d'intégration & système
 - Tests d'acceptation
 - Intégration continue (CI)
 - Développement orienté par le test (TDD)

TABLE DES MATIÈRES

Définitions & concepts

Conception des tests

Place du test dans le dev

Tests unitaires

Tests de couverture

Qualimétrie

Tests de performance

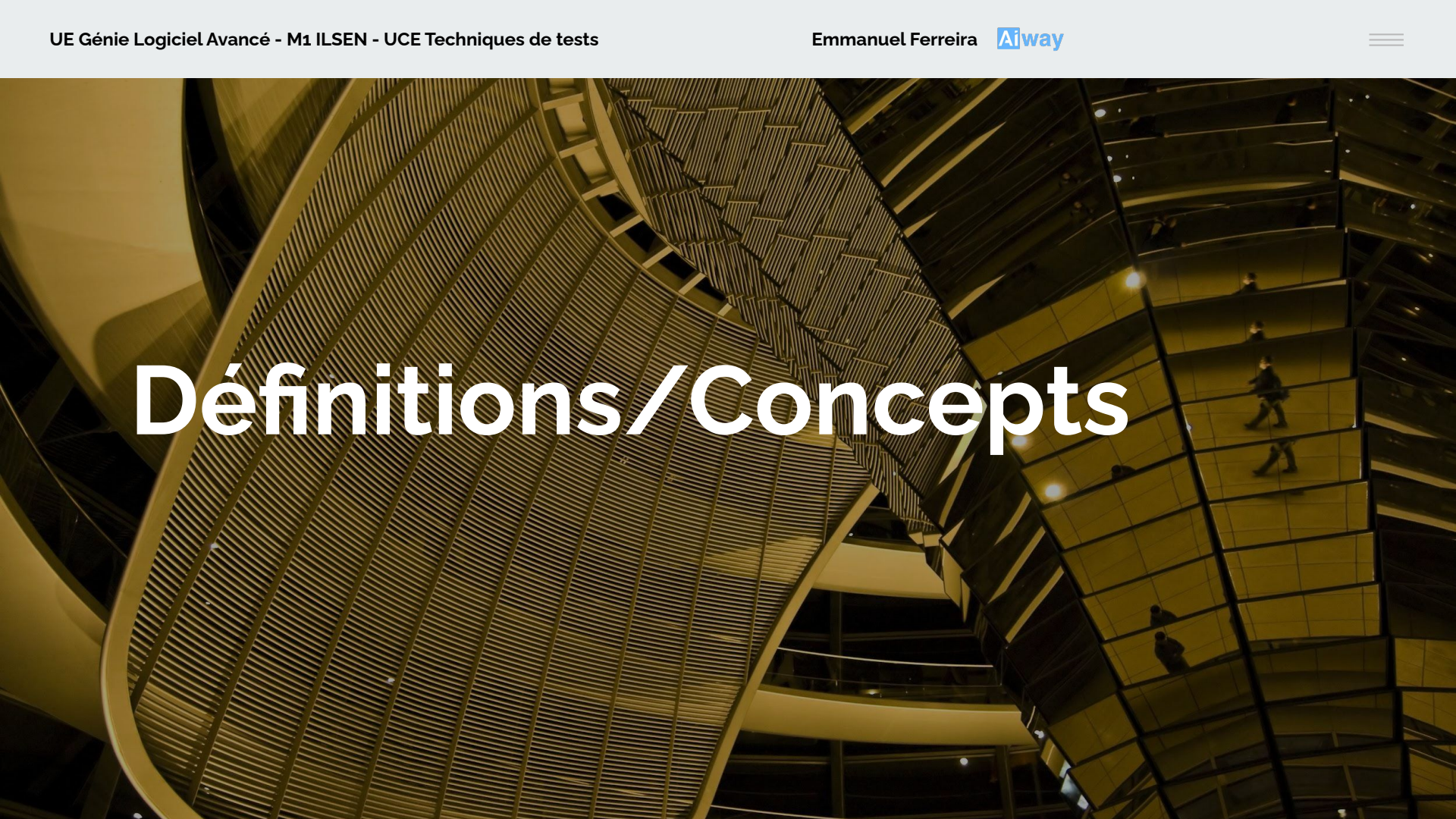
Tests de sécurité

Tests d'intégration & système

Tests d'acceptation

Intégration continue

Vers toujours plus d'automatisation



Définitions/Concepts



Le test c'est quoi ?

Testing is the process of executing a program with the intent of finding errors
The Art of Software Testing - Myers, Sandler and Badgett



Un **test** désigne une procédure dont l'objectif principal est d'identifier un nombre maximal de comportements problématiques du logiciel

Corrections des problèmes identifiés ⇒ augmentation de la qualité

Toutes les activités qui consistent à rechercher des informations quant à la qualité du système afin de permettre la prise de décisions

Pourquoi tester ?

- Les logiciels sont productions humaines « directes »
 - La communication entre humains est faillible
 - Besoins, Exigences et Code
 - Les humains sont faillibles
 - Erreurs, oubli, malentendu, maladresse, méprise...
- Les logiciels présentent des défauts
- Les conséquences de ces défauts
 - Pertes de temps, pertes de réputation, pertes financières
 - Blessures ou décès
 - « Le logiciel qui pilotait le Therac-25 ne détectait pas ce mauvais fonctionnement qui engendrait une irradiation massive et potentiellement mortelle pour le patient. »
(source : wikipedia)



Le Vol 501 d'Ariane 5

- Vol inaugural du lanceur européen Ariane 5 (1996)
 - Échec, causé par un dysfonctionnement informatique
 - Explosion en vol seulement 36,7 secondes après le décollage
 - Perte de 4 satellites valeur totale de 370 millions de dollars
- Analyse du dysfonctionnement informatique
 - Variable accélération horizontale Ariane 4 : 8 bits
 - Ariane 5 plus puissante et brutale : 9 bits nécessaires
 - Dépassement 8 bits, donc une valeur absurde dans la variable
 - Effet domino, le logiciel décide de l'autodestruction de la fusée !
- Logiciel non testé pour Ariane 5 par mesure d'économie



Cloud "Windows Azure" et 2012

- Indisponible pendant plusieurs heures le 29 février
 - La panne a débuté le matin à 1h45
 - A 13h30, Microsoft lance un correctif sur les systèmes
 - Le problème est clos tard le lendemain matin à 6h
- Cause de la panne
 - Bug lié à une erreur de calcul de date
 - Année bissextile 29 jours et non 28 jours
 - Un certificat SSL incompatible avec le jour supplémentaire
- Premier passage d'une année bissextile
 - 2010 Disponibilité commerciale (US)



Windows Azure

USS Yorktown (CG-48)

- Navire de guerre US Croiseur lance-missiles
 - Lancé en 1984 et désarmé en 2004
 - 173 mètres de long, 9 800 tonnes, vitesse 60 km/h
 - 4 × turbines à gaz puissance totale 60 000 kW
- 1996 Banc d'essai du navire intelligent
 - Objectif réduire de 10% l'effectif du navire
 - Économie prévue : \$2,8 millions par an
- Une division par zéro le transforme en épave flottante
 - Membre d'équipage rentre un 0 dans le système
 - « Réseau » tombe : défaillance du système de propulsion





Les 7 principes des tests

01

- Les tests montrent la présence de défauts
 - Mais ne peuvent en prouver l'absence ([Edsger W Dijkstra](#))
- Les tests exhaustifs sont impossibles
 - Tester toutes combinaisons d'entrée et de pré-conditions
 - Choix de la couverture des tests
 - Analyser les risques et les priorités
 - Déterminer où porter l'effort de tests
- Tester tôt
 - Plus les défauts sont détectés tôt moins ils sont coûteux à corriger



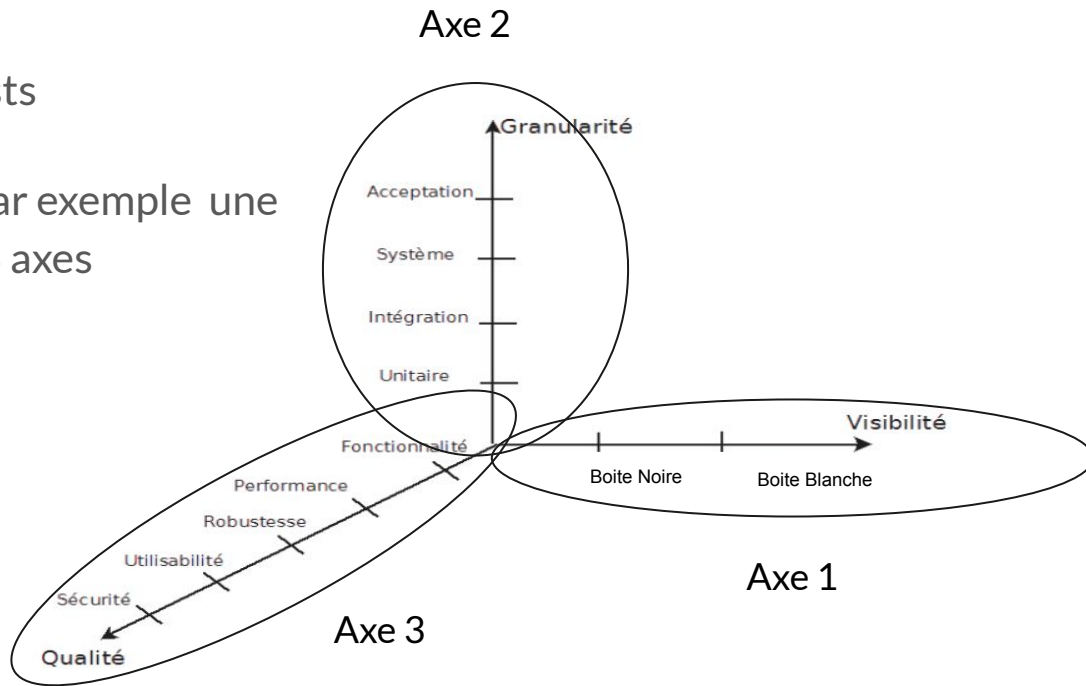
Les 7 principes des tests

02

- Regroupement des défauts
 - Règle des « 80/20 » : 80 % des défauts sont localisés dans 20 % du logiciel
- Paradoxe du pesticide
 - Le code « s'adaptent » aux tests ⇒ apporter des changements réguliers
- Les tests dépendent du contexte
 - Testeriez-vous de la même façon une application de gestion de prêts de livres et un logiciel de gestion du trafic aérien ?
- L'illusion de l'absence d'erreurs
 - Produit est mal conçu : trouver et corriger des bugs ne résout pas le fond du problème
 - Spécifications vs utilisation
 - Solution : augmenter la variété des tests appliqués

Classification des tests

- Grandes variétés tests
 - Test = mot valise
- Tretmans propose par exemple une classification selon 3 axes

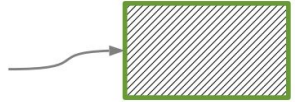


Axe 1 : Visibilité

- « **Boîte noire** » - Méthodes fonctionnelles

- Pour des entrées et des conditions d'exécution données $\Rightarrow$ vérifie si les sorties d'un système sont bien celles prévues
- Ignore les mécanismes internes d'un système $\Rightarrow$ centrées sur les exigences

Boîte
noire



- « **Boîte blanche** » - Méthodes structurelles

- Test sur un système dont la structure interne est connue
- Centrées sur le fonctionnement (et non sur la fonctionnalité)
 - Logique interne
 - Justesse du code

Code Source

Boîte blanche



- Existe un entre deux ~ « **Boîte Grise** » :

- Exemple : l'architecture (relations) des modules d'un système est connue, mais pas le code de chaque module



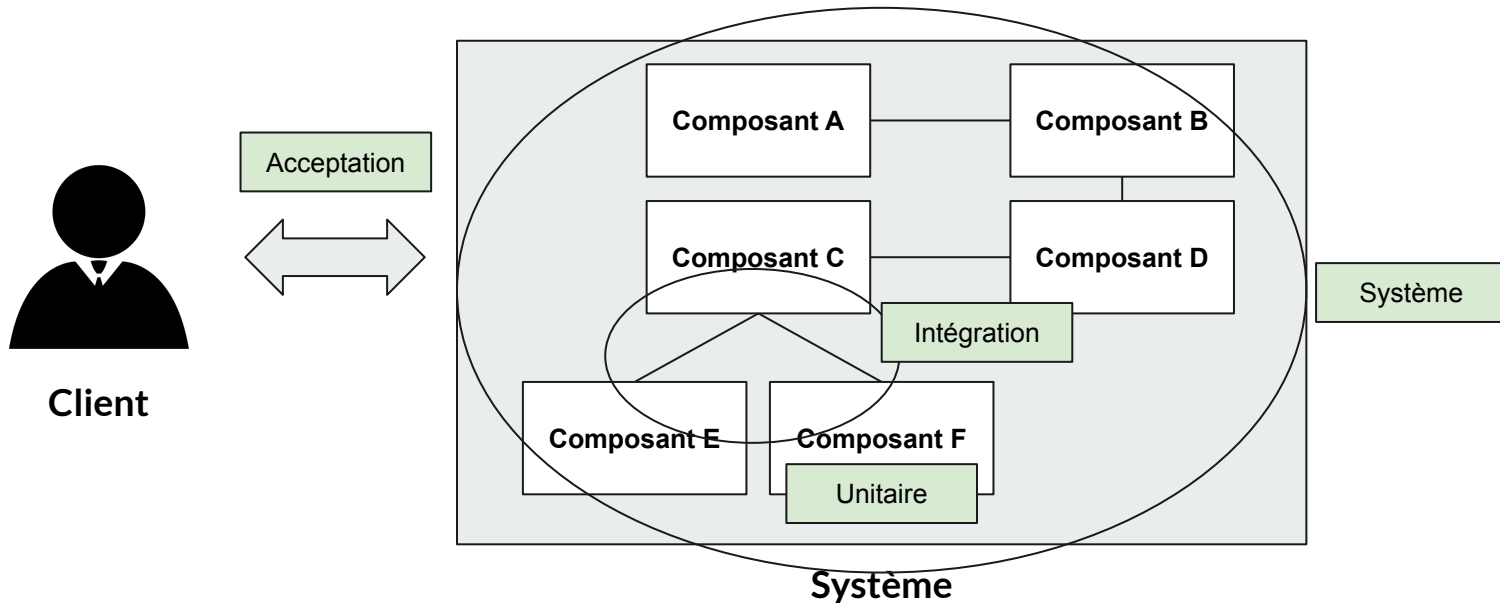
Axe 2 : Granularité/Niveau d'abstraction

01

- Tests de composants ou tests unitaires
 - Chaque composant « critique » est testé de manière séparée
- Tests d'intégration
 - Vérifier que les composants interagissent correctement entre eux
 - Gestion des dépendances entre composants
- Tests système
 - Tester le comportement du système dans son ensemble
- Tests d'acceptation (tests de recette)
 - Précèdent la mise en oeuvre opérationnelle
 - Alpha-testing (équipe de développement)
 - Beta-testing (panel d'utilisateurs finaux - scénarios)

Axe 2 : Granularité/Niveau d'abstraction

02





Axe 3 : Selon la qualité/l'attribut testé

01

- Test de fonctionnalité / conformité
 - Tester ce que « fait » le système, le composant
 - Est-il conforme aux spécifications fonctionnelles/exigences ? (boîte noire)
- Test de performance
 - Évaluer la capacité d'un système/composant à fonctionner correctement sous contraintes de temps et de ressources
 - Montée en charge
- Test de robustesse
 - Évaluer la capacité d'un système/composant à fonctionner correctement en présence d'entrées invalides ou de conditions d'utilisation stressantes
 - Tolérance aux incidents



Axe 3 : Selon la qualité/l'attribut testé

02

- Test d'utilisabilité
 - Mesurer la faciliter d'utilisation et de prise en main du système
 - Problèmes d'ergonomie, niveau de documentation
 - Observer directement l'utilisateur en train de se servir de l'application ou du logiciel
- Test de sécurité
 - Mesurer le niveau de vulnérabilité du système
- + Test Meta (e.g. test des tests)
 - Exemple = Couverture
 - % par rapport à la structure ⇒ Boîte blanche



Autre classification : statique /dynamique

- Statique :

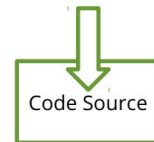
- Qualimétrie

- Déclarations, utilisation des variables, respect du typage, etc.
 - Respect de règles de codage (coding style)
 - Calcul de métriques (complexité, dépendances...)

- Obligatoirement en boîte blanche

- Complémentaire aux activités “manuelles” de vérification du code source

- Recherche des défauts simples et fréquents (contrôle)
 - Revue de code, inspections, pair-programming



- Dynamique :

- Obligatoirement code exécutable





Autre classification : manuel / automatisé

- Test Manuel
 - La manière la plus directe de faire des tests
 - Auto-adaptable/évolutif (flexibilité) ET parfois indispensable (jugement humain / réaction)
 - MAIS activité qui peut être lourde et répétitive (temps, fiabilité, reproductibilité, traçabilité)
 - Niveau humain (pour le meilleur et pour le pire)
- Test Automatisé
 - Utilitaires permettant d'exécuter et d'assister les activités liées aux tests
 - Conception, exécution et suivi des tests (rapport)
 - Répétabilité et fiabilité
 - Tout n'est pas (encore ?) automatisable...
 - Simuler/anticiper la perception, les comportements des utilisateurs, etc.



Les (mauvaises) réticences à l'automatisation

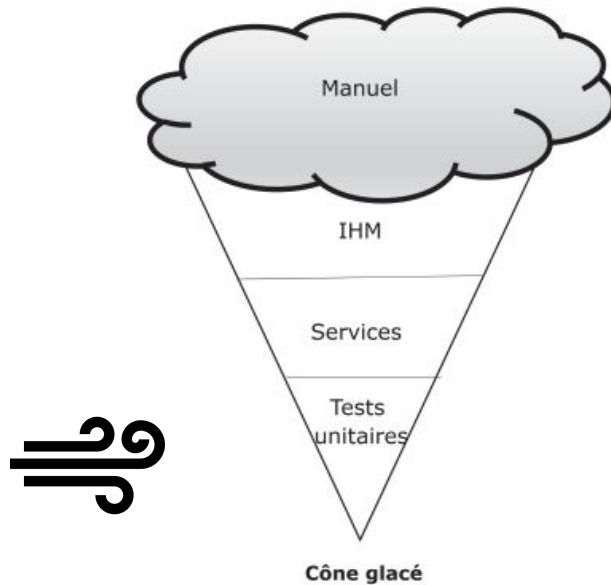
- Ça prend trop de temps d'écrire des tests !
- Ça prend trop de temps d'exécuter des tests !
- Ce n'est pas mon boulot de tester mon code !
- Je suis payé pour écrire du code, pas pour le tester !
- Je ne sais pas faire (ou pas “bien”) !



Choix de l'automatisation

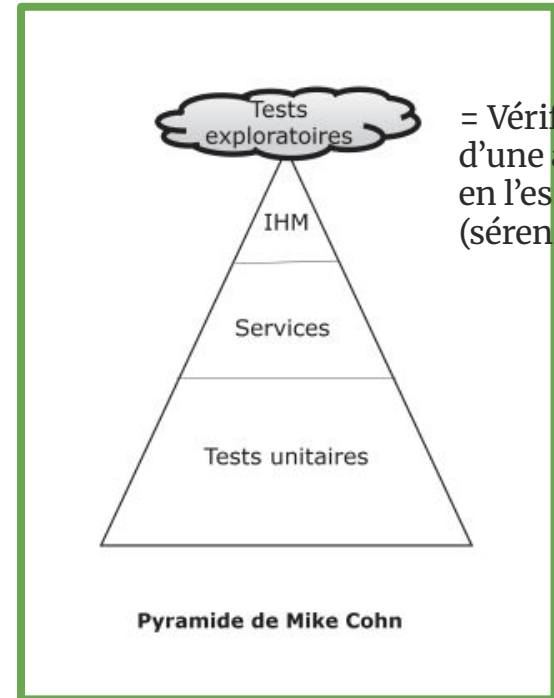
- Tendance générale $\Rightarrow$ augmenter la part couverte par l'automatisation
 - Surtout valable pour les tests unitaires, d'intégration, et système
 - Assisté par la diminution du surcoût de création
 - Fonction de la taille, de la nature et de la criticité d'un projet
 - Artefacts de tests (codes, fichiers de configuration, jeux de données) participent à la documentation du logiciel
- Choix en fonction d'une stratégie et selon des critères
 - Faisabilité ?
 - Coût + gains de l'automatisation VS Coût de la répétition manuelle ? Risques d'erreurs ?
 - Test important/vital ? relevant d'une bonne pratique ?
 - Besoin en répétition
 - Motivé par la méthode développement choisie ?

Stratégie d'automatisation des tests



Anti-pattern (test par interface utilisateur)

VS



= Vérifier la qualité d'une application en l'essayant (sérendipité)



Outils de tests

- Des outils existent pour toutes les activités de test
 - Aide à la gestion du test et des tests
 - Aide aux tests statiques
 - Aide à la spécification des tests
 - Aide à l'exécution et enregistrement des résultats des tests
 - Support de performance et surveillance
- Pour le développement en particulier
 - Outils d'analyse statique
 - Framework de tests unitaires
 - Outils de mesure de couverture
 - Outils d'analyse dynamique

En pratique qui teste ?

- Développeur
- Testeur (ou ingénieur qualité)
- Utilisateur (Tests d'acceptation)
- Tiers (indépendants) - certification
- Mais surtout vous à partir d'aujourd'hui ^^





Conception des tests



Tester c'est décider

- Deux contraintes antagonistes
 - Le logiciel doit comporter le moins de défaut possible
 - Contrainte d'acceptabilité par le client
 - L'effort de test doit être le plus faible possible
 - Contrainte de délais, de ressources, de budget/coût
- Il faut donc décider de manière pertinente
 - Des composants, sous-systèmes à tester
 - Des types de tests à mettre en œuvre
 - De la part d'automatisation raisonnable
 - Des outils à utiliser pour l'automatisation



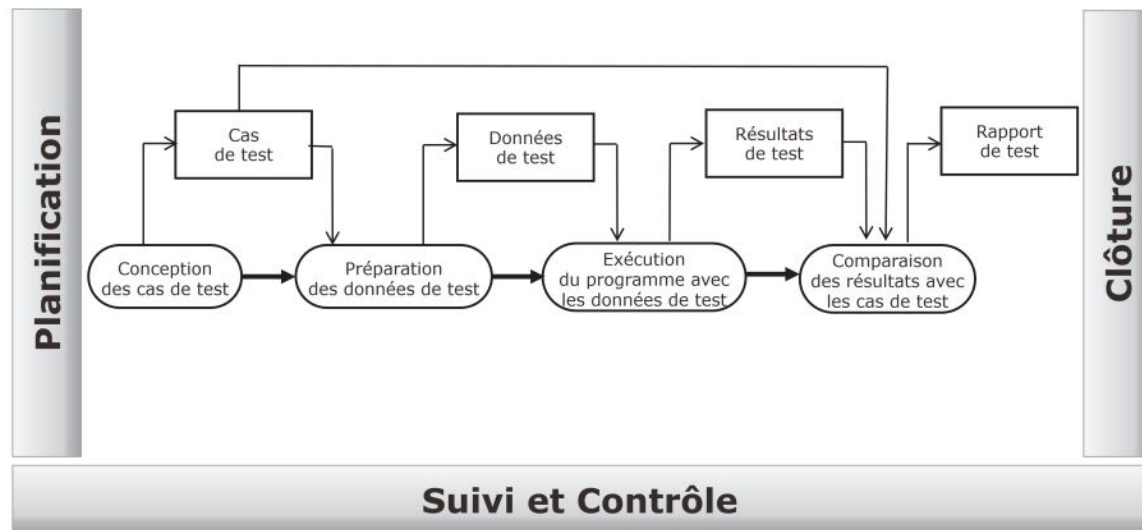
Processus générique de test

01

- Organisation et Planification
 - Définir la stratégie qui sera mise en place - **plan de test**
 - Sélection pertinente /projet
- Analyse et conception
 - Rédiger un ou plusieurs **cas de test** selon la stratégie
 - Données (jeu de test), un **scénario** à exécuter, des **résultats attendus**.
 - On parle de **suite de test** si plusieurs cas de test
- Implémentation et exécution
 - Écrire et exécuter des « scripts » de tests
 - Signaler et analyser les divergences entre attendus et obtenus
- Bilan
 - Établir une synthèse de la phase de test + stockage (traçabilité)
 - Apporter éventuellement des préconisations

Processus générique de test

02





Plan de test

- La stratégie dépend :
 - de la criticité du logiciel et plus largement du plan qualité (exigences)
 - du coût de développement
- Une stratégie définit :
 - Ressources mises en œuvre (équipes, testeurs, outils, etc.)
 - Mécanismes du processus de test (gestion de configuration, évaluation, etc.)
- Une stratégie tient compte :
 - Du type d'application cible (temps réel, protocole, base de données...)
 - Des méthodes de spécification & de conception retenues
 - Langages de programmation utilisés
 - Expérience de l'équipe projet



Cas de test

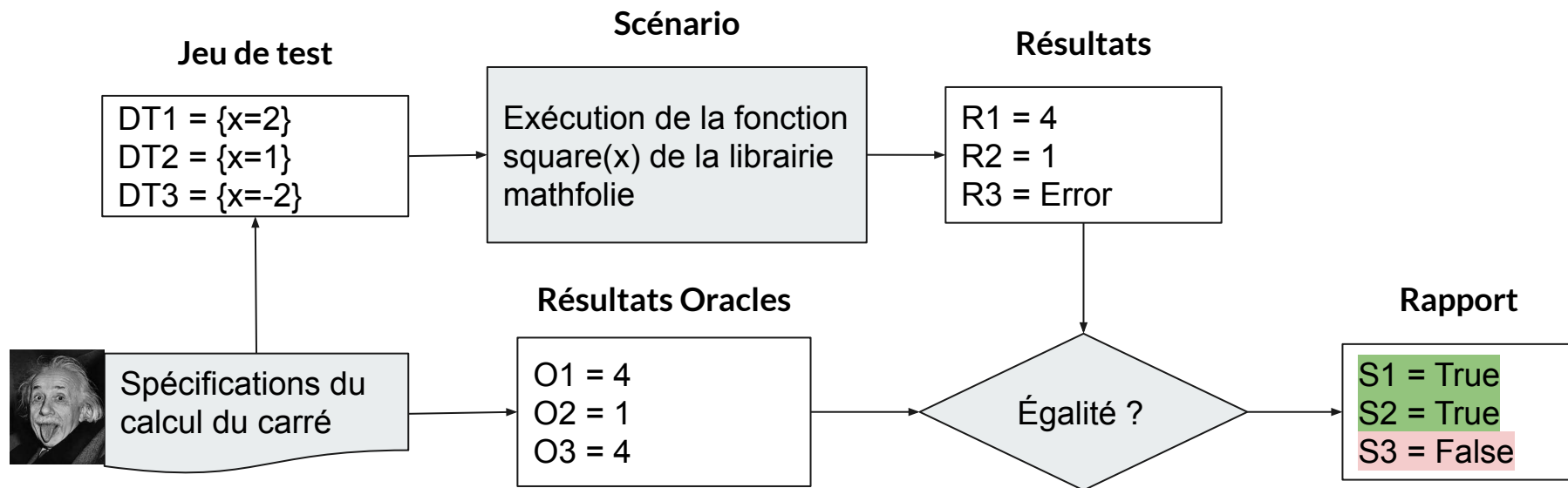
01

- Cas de test = <jeu de test, scénario, résultat oracle>
 - Jeu de test : données qui permettront de réaliser le test
 - Données de test (DT)
 - Entrées qui seront fournies au logiciel (ou une partie) pendant l'exécution dynamique du test
 - Scénario de test : actions à effectuer (ensemble d'instructions)
 - Le scénario de test produit un résultat
 - Ce résultat doit être comparé de manière manuelle ou automatique à une référence (un oracle)
 - Référence = résultat attendu par rapport aux spécifications (pas /code) !
- Gardes fous :
 - Faire tester par un autre développeur que celui du code sous test
 - Ou les implémenter avant (cf. TDD)

Cas de test - un petit exemple

02

Soit le test d'une fonction permettant de calculer le carré d'un nombre





Définition du jeu de test

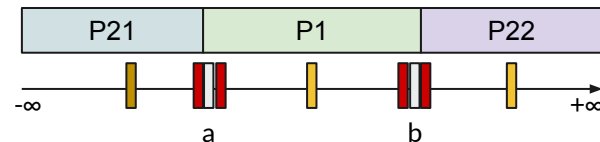
01

- Exhaustivité des tests bien souvent impossible en pratique
- Néanmoins il est important de vérifier que le programme “marche” dans des cas valides comme dans les cas invalides
 - Si une exception doit être levée : il faut vérifier qu'elle l'est !
- Dans cette démarche d'identification du jeu de test, deux notions clés :
 - **Partition d'équivalence**
 - Une portion d'un domaine d'entrée ou de sortie pour laquelle le comportement d'un composant ou système est supposé être le même
 - **Valeur limite**
 - Une valeur d'entrée ou de sortie qui est aux bords des partitions d'équivalence
 - Exactement aux bornes
 - Juste en dessous/au dessus (plus petit incrément) des bornes des partitions

Définition du jeu de test - exemple

02

- Soit
 - F : la fonction doit retourner vrai si un entier x est compris dans un intervalle (inclusif) défini par les nombres a et b (avec a inférieur à b)
- Les partitions d'équivalence pour x
 - x est compris entre a et b : $P1$
 - x n'est pas compris entre a et b : $P2$
 - $P2$ peut être partitionné à son tour en
 - x est strictement inférieur à a : $P21$
 - x est strictement supérieur à b : $P22$
- Les valeurs limites pour x
 - x est égal à a et x est égale à b
 - Mais aussi les $a-1/b-1$ et $a+1/b+1$





Place du test dans le développement



Quand tester ?

- À chaque niveau du cycle de vie du logiciel
 - Valable pour les différents modèles de développement
 - Cycle en V
 - Méthodes agiles : Test Driven Development (TDD)
- En pratique :
 - Après avoir écrit votre code fonctionnel
 - Le danger ici est de se laisser aller à l'idée que finalement écrire un (ou plusieurs) test(s) n'est pas nécessaire, par manque de temps ou par excès de confiance
 - Lorsqu'un bug est détecté (effet boule de neige)
 - Avant d'écrire votre code fonctionnel (cf. TDD)



Rôles du test - Exercice

01

- En pratique un même test peut jouer plusieurs rôles dans le cycle de vie du logiciel
- Soit la situation suivante :
 - Une version « initiale » 2.8.0 est testée avec une suite de test SA qui comprend les cas de test SAT1, SAT2, SAT3, SAT4.
 - Deux défauts sont détectés sur SAT2 et SAT3 que corrige la version 2.8.1. Elle est testée, les défauts sont corrigés mais SAT4 donne un résultat négatif.
 - Après test, il s'avère que tous les défauts sont corrigés dans la 2.8.2.
 - Une nouvelle version 2.9.0 qui intègre de nouvelles fonctionnalités est testée, avec la suite de test SB qui comprend les cas de test SBT1, SBT2, SBT3.
- Présentez sous forme de tableau, les versions, tests, rôles et résultats des tests
 - Notation : (D|C|R) / (S|E|?) ⇔ (Détection/Correction/Régression) / (Succès/Échec/Inconnu)
 - Exemples : D / S, R / E ou encore R / ?

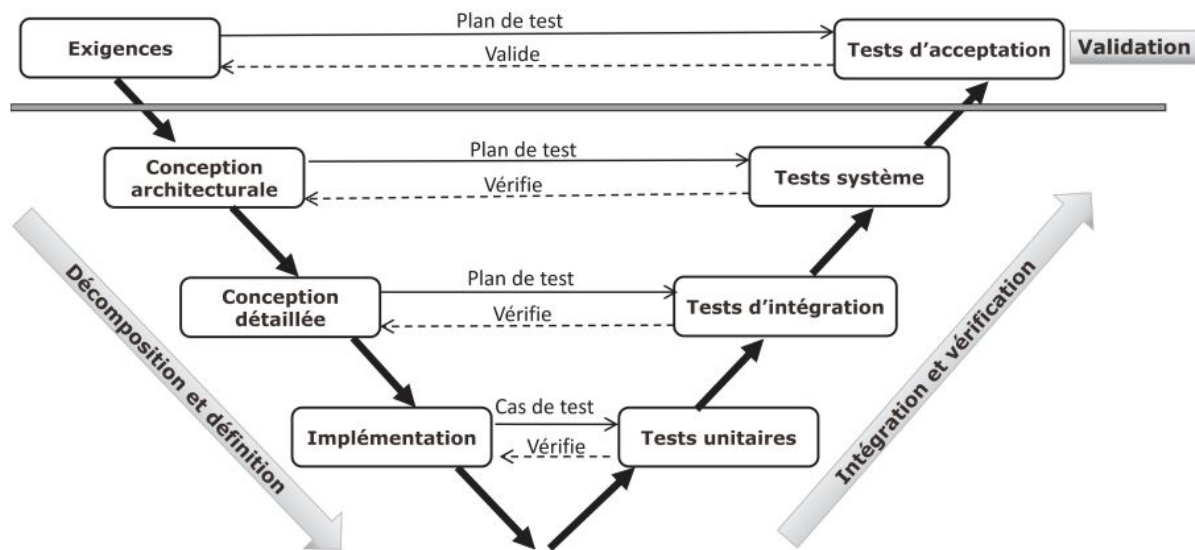


Rôles du test - Exercice

02

Suite	Test	2.8.0	2.8.1	2.8.2	2.9.0
SA	T1	D / S	R / S	R / S	R / ?
	T2	D / E	C / S	R / S	R / ?
	T3	D / E	C / S	R / S	R / ?
	T4	D / S	R / E	C / S	R / ?
SB	T1				D / ?
	T2				D / ?
	T3				D / ?

Cycle de développement en V





Méthodes agiles

- Basées sur des cycles courts de développement
 - Retours réguliers et fréquents (le feedback)
 - Itératifs et incrémentaux
- + Ensemble de bonnes pratiques qui interagissent les unes avec les autres
- Complémentaires :
 - Pratiques orientées processus (e.g. Scrum, Kanban)
 - Pratiques orientées technique (e.g. eXtreme Programming)
 - Tests comme instrument de conception (test first)

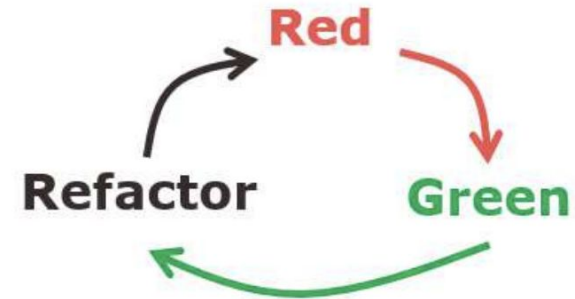


Test Driven Development (TDD)

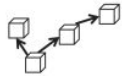
- Approche itérative et incrémentale de codage piloté par les tests unitaires
- Changement de paradigme :
 - Tests sont écrits juste avant le code, qui lui, une fois écrit, se doit de faire passer les tests
 - Objectif ⇒ produire rapidement un code opérationnel puis propre (cf. slide suivant)
- Un standard de développement aujourd'hui :
 - Réduction considérable des risques de régression
 - Guide les développeurs à mieux écrire le code des applications.
 - Connaître les résultats finaux attendus permet de partir dans la bonne direction dès le démarrage des développements

Mantra du TDD - Cycle Red-Green-Refactor

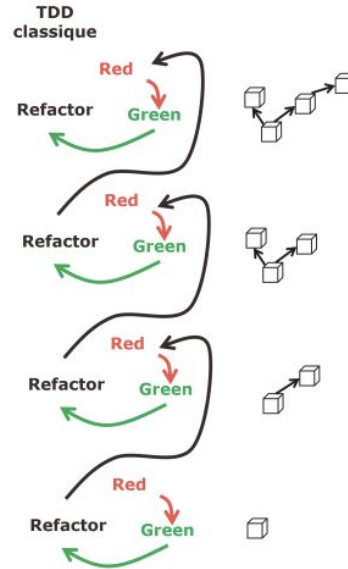
- **Red** = Écrire un nouveau test unitaire en échec
 - Test $\Leftrightarrow$ nouveau comportement à implémenter
- **Green** = écrire au plus vite un code pour faire passer le test précédent ainsi que les tests antérieurs
 - Tous les moyens sont bons $\Rightarrow$ y compris de mauvaises pratiques (duplication, patch, complexification, etc.)
- **Refactor** = temps spécifique consacré au refactoring
 - Changer la structure interne d'un logiciel sans en changer son comportement observable
 - Respect des bonnes pratiques



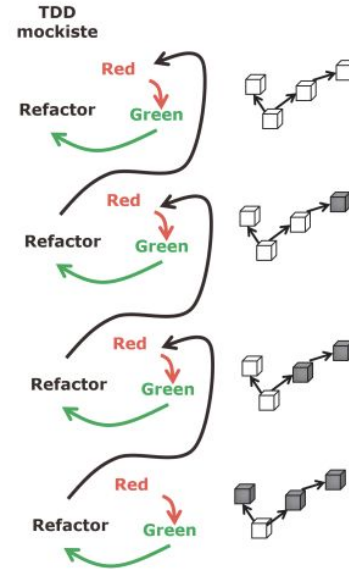
Quid d'un système complexe ?



Cible



Option 1



Option 2 = TDD mockiste

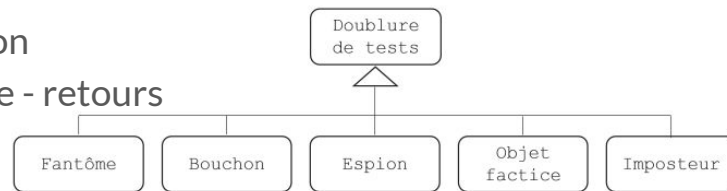
Avec

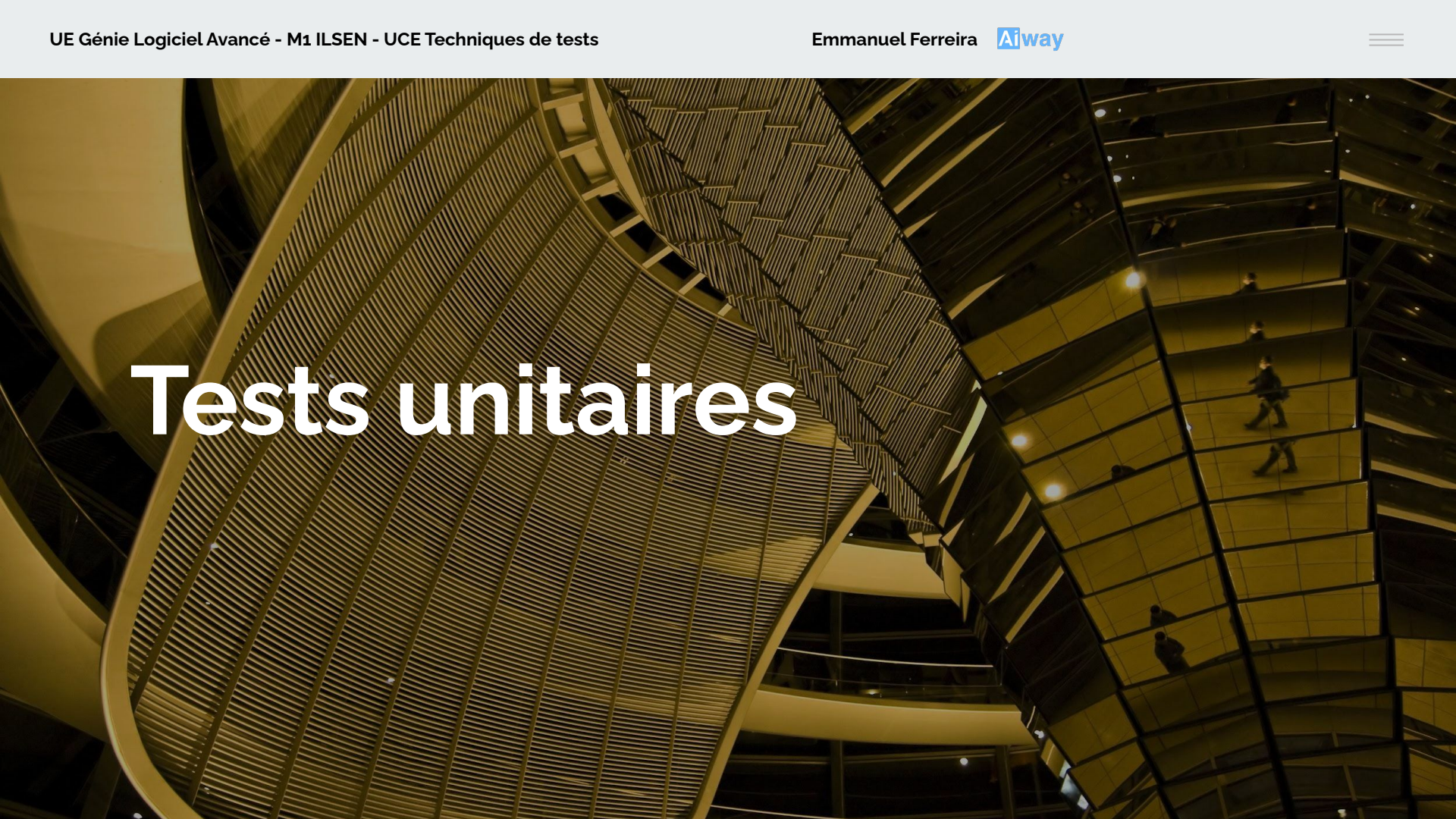


Doublure

Doublure : définition

- Artifice qui permet de reproduire l'état et/ou le comportement du composant dont dépend le code à tester.
- 5 types
 - Le **fantôme** (dummy) : aucune implémentation
 - Le **bouchon** (stub) : implémentation minimale - retours
 - L'**espion** (spy) : bouchon avec des logs
 - L'**objet factice** (mock) : Objet simulé
 - Comportement décrit spécifiquement pour/dans un test unitaire
 - Vérifie la validité et l'enchaînement d'appels de méthode sur l'objet simulé
 - L'**imposteur** (fake)
 - Implémentation alternative opérationnelle - simplifiant des dépendances
 - Exemple : base de données en mémoire au lieu d'une base de données réelle.





Tests unitaires



L'objectif d'un test unitaire

- S'assurer du bon fonctionnement d'une **unité** de code (plus petit élément identifiable)
 - Dépend du paradigme de programmation utilisé
 - Procédural : chaque procédure / POO : pour chaque classe chaque méthode exposée
- Vise à identifier les défauts des unités en les testant **séparément** (isolation)
 - Doivent pouvoir être lancés dans n'importe quel ordre sans en affecter leurs résultats
- Doivent pouvoir être lancés très fréquemment ⇒ **rapides & automatisés**
 - Idéalement après chaque modification du code
- Doivent être **idempotents** :
 - Quel que soit le nombre de fois où un test est joué, il doit produire le même résultat
- Dans la pratique, bien souvent mis en place par les développeurs eux-mêmes pour :
 - Vérifier la bonne exécution des fonctions/classes/modules développées
 - Mieux comprendre les attentes ⇒ approche TDD (avant même la première ligne de code)

Tests unitaires en pratique

- En POO, pour chaque classe testée : une ou plusieurs classes contenant une suite de test, avec pour chaque méthode un ou plusieurs cas de test
- Boîte noire (interfaces) mais surtout boîte blanche (appels aux méthodes privées)
- Que faire si le comportement dépend d'autres unités ?
 - Réponse A : on ne teste pas unitairement 😞
 - Réponse B : on teste en utilisant les autres unités si elles sont disponibles
 - Q : Mais alors c'est plutôt du test d'intégration non ?
 - R: oui... 😞
 - Réponse C : on simule le comportement des autres unités par une doublure ! 😊
 - Réponse D : la réponse D 😞
- Que faire si le comportement dépend d'éléments non contrôlables ? (ex réseau, base de données, aléatoire, etc) ⇒ On les simule aussi !

Framework de test unitaire ~ xUnit

- Test base : très bien outillé aujourd'hui et intégré dans les IDE
- Pour Java
 - JUnit
- Pour Python
 - unittest (standard framework), **pytest**
- Pour PHP
 - PHP Unit, SimpleTest
- Pour C++
 - cppunit, Google Test

```
public final class SumTest {  
  
    @Test  
    public void testSum() {  
        final int sum = 2 + 2;  
        assertEquals(4, sum);  
    }  
  
}
```





Classe de test (JUnit)

- Classe déclarant des méthodes publiques sans paramètre et sans valeur de retour et qui sont annotées par `@Test` ⇒ méthodes de test
- Selon le modèle AAA - une méthode de test doit contenir les phases suivantes :
 - Arrange : création des objets, des données de test et définition des attentes
 - Act : invocation de la méthode testée)
 - Assert : résultat du test unitaire
 - Vérifications (assertTrue, assertEquals) entre résultat attendu (oracle) et résultat obtenu
- De plus, elle ne devrait pas contenir d'instruction if ou switch puisqu'un test traduit un cas d'utilisation simple sans choix possible.
- De même, une méthode de test ne devrait contenir qu'exceptionnellement des boucles for ou while.



Petit exemple simpliste

```
import static org.junit.Assert.*;
import org.junit.Test;

public class StringTest {

    @Test
    public void upperCaseProduitUneChaineEnMajuscules() throws Exception {
        // Bloc arrange
        String s = "Bonjour le monde";

        // Bloc act
        String maj = s.toUpperCase();

        // Bloc assert
        assertEquals("BONJOUR LE MONDE", maj);
    }
}
```



Fixtures

- Pour réaliser un test, il est parfois nécessaire de créer un grand nombre d'objets et de préparer le SUT (System Under Test)
 - Plutôt que d'écrire le code nécessaire dans le test (au risque de le rendre moins lisible, ou encore de dupliquer du code), on préfère écrire ce code dans des méthodes à part ⇒ on parle alors de **fixtures**.
- Avec JUnit, il est possible d'exécuter des méthodes avant et après chaque test pour allouer et désallouer des ressources nécessaires à l'exécution des tests.
 - On déclare pour cela des méthodes publiques sans paramètre annotées avec `@Before` ou `@After`
 - NB : dans d'autres framework on peut retrouver la notion de `setUp` et `TearDown`



Petit exemple de fixture (JUnit)

```
import org.junit.After;
import org.junit.Before;
import org.junit.Test;
import static org.junit.Assert.*;

public class UneClasseTest {

    @Before
    public void initTestEnvironment() {
        // cette méthode est exécutée avant chaque test
    }

    @After
    public void destroyTestEnvironment() {
        // cette méthode est exécutée après chaque test
    }

    @Test
    public void methodeDeTest() throws Exception {
        // la méthode de test
    }
}
```



Tester les exceptions

- Grâce aux tests unitaires, il est également possible de tester les cas non nominaux qui se traduisent la plupart du temps par la production d'une exception en Java
- Si on désire tester un cas d'erreur par exemple, cela signifie que le test sera en succès si une exception précise est produite lors de la phase Act



Petit exemple de test d'exception (JUnit)

```
import org.junit.Test;

public class IntegerTest {

    @Test
    public void parseIntThrowsExceptionWhenNotANumber() throws Exception {
        assertThrows(NumberFormatException.class, () -> {
            Integer.parseInt("not a number");
        });
    }
}
```



Framework pour construire des doublures

- Java
 - Mockito
- Pour PHP
 - Mockery
- Pour Python
 - unittest.mock
- Pour C++
 - Google Mock





Mocks avec Mockito

- Mockito permet création d'objets mocks à partir d'une classe ou d'une interface.
- La méthode statique `Mockito.mock(Class<?>)` permet de créer une instance d'un mock à partir d'une classe ou d'une interface.
- L'instance d'objet retournée par cette méthode est instrumentalisée par Mockito.
- Il est possible d'enregistrer sur ce mock des comportements lors de la phase Arrange (ou Fixture) grâce, notamment, à la méthode `Mockito.when(Object)`.
- Lors de la phase Assert, il est possible de vérifier que les appels de méthodes programmés sur le mock ont bien été réalisés grâce à la méthode `Mockito.verify(Object)`.



Petit exemple d'utilisation de Mockito

```
import static org.junit.Assert.*;
import javax.servlet.http.HttpServletRequest;

import org.junit.Test;
import org.mockito.Mockito;

public class TestMyClient {

    @Test
    public void testGetLoginWithMockito() throws Exception {
        HttpServletRequest mockedRequest = Mockito.mock(HttpServletRequest.class);
        Mockito.when(mockedRequest.getParameter("login")).thenReturn("monlogin");
        client c = MyClient(mockedRequest)

        String parameterValue = c.getLogin();

        assertEquals("monlogin", parameterValue);
        Mockito.verify(mockedRequest).getParameter("login");
    }
}
```



Écrire des programmes testables (POO)

- Généralement, on groupe dans une classe les tests ayant la même classe comme point d'entrée
 - Par **convention** on nomme la classe de test à partir du nom de la classe testée préfixé ou suffixé par Test
 - Exemple, pour tester la classe Math, on créera une classe TestMath ou MathTest
 - Très utile pour la découverte automatique des tests via un IDE
- La classe de test étant par nature extérieure à la classe testée
 - Il faut donc pouvoir :
 - Accéder simplement à l'état d'un objet;
 - Amener un objet dans un état propice au test
- Ne pas hésiter devant le refactoring
 - Tester amène souvent à revoir son code en l'améliorant...



Tests unitaires : que tester ?

01

- Petite piqûre de rappel (Pfizer ?)
 - Découper le comportement de la méthode en sous comportements (partitions d'équivalences) qu'il faudra tester individuellement :
 - Dans tel cas, la méthode doit lever une exception;
 - À tester aussi !
 - Dans tel autre cas, elle doit retourner une valeur particulière
- Conditions aux limites :
 - Les problèmes sont souvent là !



Tests unitaires : que tester ?

02

- Conformité :
 - Est-ce que la donnée de sortie est conforme à un format prédéfini ?
- Ordre :
 - Dans le cas où on travaille sur une collection ordonnée :
 - Si on cherche une valeur : vérifier qu'on la trouve bien en tête/milieu/fin de collection
 - Si une méthode prend une collection en entrée :
 - Le code présuppose-t-il un ordre particulier pour cette collection ?
 - Si une donnée interne doit être maintenue triée, le vérifier ;
- Intervalle
 - Tester les valeurs litigieuses : la plus petite valeur possible, la plus grande, etc.
 - L'intervalle possible peut être parfois plus grand que celui qui est visé conceptuellement (un âge codé sur un entier par exemple)



Tests unitaires : que tester ?

03

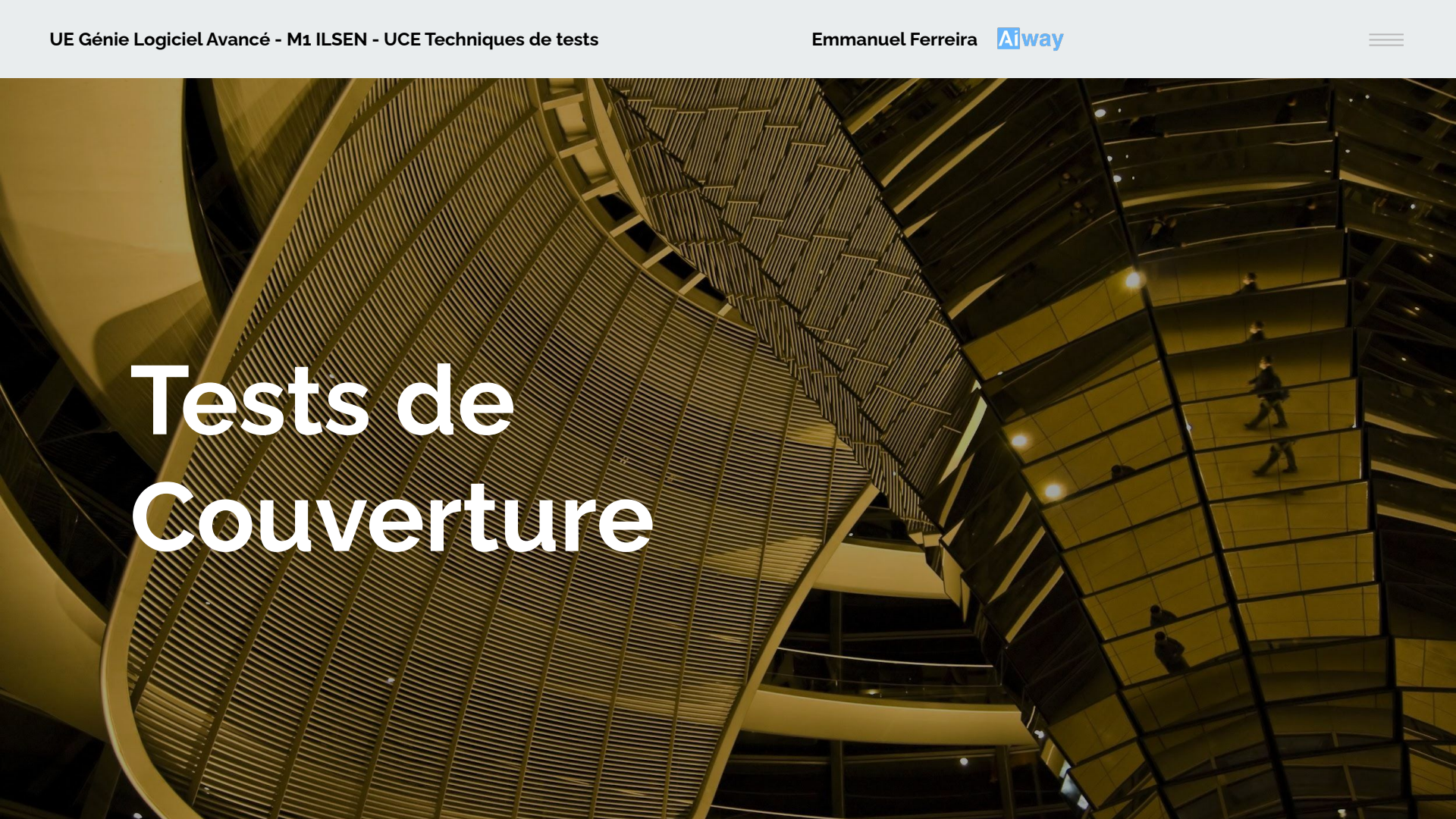
- Référence
 - Cas où votre méthode référence d'autres méthodes ou classes : tester les conditions où elles sont utilisées
- Cardinalité
 - Par exemple si on doit maintenir et publier un top-10 des meilleures ventes :
 - Peut-on publier un top-ten vide ? à un élément ? à moins de 10 ? à 10 ?
- Date
 - Quel calendrier ? Changements d'heures ?
- Symétrie et fonction inverse
 - Si on calcule une racine carrée, vérifier que le résultat élevé au carré donne la valeur initiale.



Tests unitaires : que tester ?

04

- Cross-checking
 - Re-calculer un résultat en utilisant une autre version (version plus ancienne abandonnée car moins efficace mais déjà testée par ex).
 - Vérifier au moyen d'invariants
 - Exemple si j'emprunte un livre j'en ai un de plus emprunté, un de moins libre, au total toujours le même nombre
- Conditions d'erreur
 - Conditions critiques (qui arrivent tout au tard) :
 - Plus de mémoire, ou d'espace disque
 - Plus de réseau
 - Base de données plantée



Tests de Couverture



Couverture : une notion polysémique

- Couverture des tests

- Part des tests attendus qui a été couverte lors du test d'un logiciel.
- Formule
 - $(\text{Nombre de scénarios de test exécutés} / \text{Nombre total des cas de test}) * 100$

- Couverture du code

- Dans quelle mesure un code source d'une application est exécuté durant l'exécution d'une suite de tests
- Formule:
 - $(\text{Nombre de lignes de code exécutées} / \text{Nombre total de lignes de code}) * 100$



La couverture du code en pratique

- Mesure qui décrit le degré auquel le code source d'un programme a été testé
 - Exécution de tous les tests (unitaires, fonctionnels, etc...) $\Rightarrow$ mesure de couverture.
 - Outils déterminent quels fragments de code ou quelles méthodes ont été exécutés.
- La mesure (ou métrique) principale
 - **Statement coverage** : rapport du nombre de lignes de code exécutées sur le nombre de lignes total du code source
- Plusieurs autres métriques (critères) existent (différents niveaux de granularité):
 - **Method coverage** : Est-ce que toutes les méthodes ont été exécutées ?
 - **Class coverage**: Est-ce que toutes les classes définies ont été employées ?
 - **File coverage**: Est-ce que tous les fichiers sources ont été utilisés ?
 - **Branch coverage** : Est-ce que les structures de contrôle (if ou while par exemple) ont été testées pour les 2 cas vrai et faux ?
 - **Path coverage** : Est-ce que tous les chemins possibles d'exécution des fonctions d'un programme ont été exécutés ?

Exemple de rapport (plugin eclemma - Eclipse)

Problems Declaration Console Coverage 

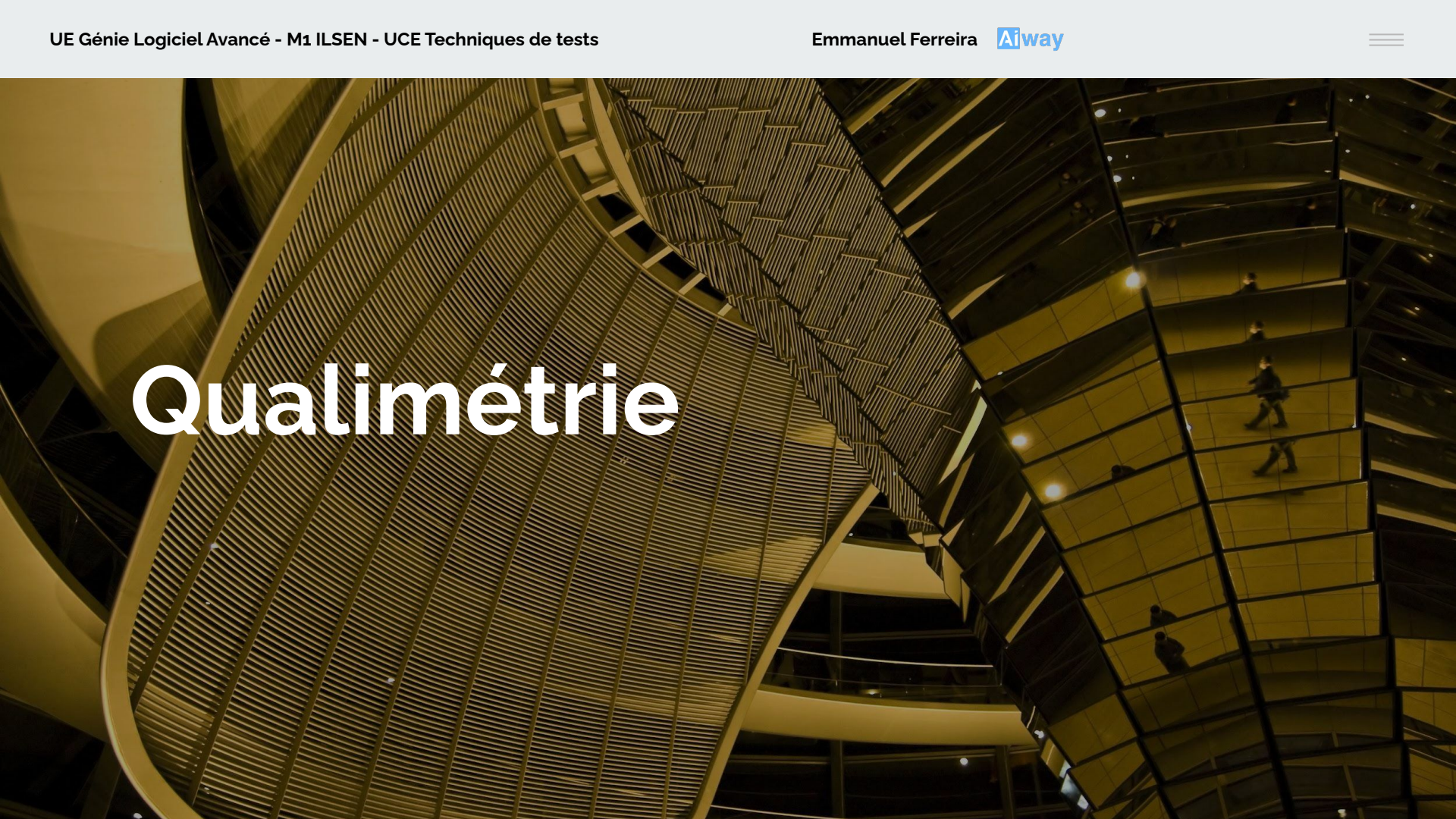
fr.koor.poo (19 avr. 2020 13:11:32)

Element	Coverage	Covered Instructi	Missed Instructic	Total Instructions
▼ Rational		182	79	261
▼ src		98	71	169
▼ fr.koor.poo		98	71	169
▼ Rational.java		98	71	169
▼ Rational		98	71	169
Rational(double)		0	29	29
toString()		0	17	17
eq(Rational)		0	15	15
Rational()		0	5	5
Rational(int)		0	5	5
Rational(int, int)		11	0	11
add(Rational)		24	0	24
getDenominatc		3	0	3
getNumerator()		3	0	3
setDenominatc		11	0	11
setNumerator()		4	0	4
simplify()		42	0	42
▼ test		84	8	92



Outils

- Java
 - JaCoCo (plugin Eclipse : eclemma)
 - Cobertura
- PHP
 - PHPUnit (composant php-code-coverage)
- Python
 - coverage
- C++
 - gcov



Qualimétrie



Qualimétrie : Définition

- Mesure la « qualité » du code source produit
- Pour s'assurer que le code respecte
 - Les normes de codage fixées (coding style) - e.g. SUN pour java, PEP 8 pour Python
 - Des bonnes pratiques “générales”
 - L'absence de copier-coller
 - L'absence de code mort (variable non utilisée...)
 - Un bon niveau de documentation
 - Présence de commentaires dans le code
- Afin de d'avoir ou de toujours avoir un code
 - Lisible, facilement maintenable et testable
- On ne s'intéresse pas à ce que fait le code MAIS à la façon dont le code est fait



Métriques de la qualité logicielle

- Travaux de chercheurs et d'acteurs de l'industrie des logiciels
- Articles de référence
 - McCabe sur la complexité du code (1976)
 - Chidamber & Kemerer sur le couplage des objets
 - Robert Martin sur l'analyse des dépendances
- Des nombreuses métriques, 3 exemples
 - La complexité cyclomatique de McCabe
 - Le couplage entre objets
 - Dépendances cycliques de packaging



Complexité cyclomatique de McCabe

- On compte le nombre de boucles et de conditions dans le code
 - On peut y ajouter le nombre d'opérateurs logique dans les conditions
- On obtient le nombre de chemins indépendants à parcourir pour traverser une méthode du début à la fin en couvrant tous les cas possibles.
- Plus la valeur obtenue est élevée, plus la méthode est complexe et difficile à comprendre.
 - 7 correspond à une méthode simple
 - 10 est le maximum acceptable
- Au delà de 10 il faut
 - Simplifier le code ou utiliser des sous-méthodes



Couplage entre objets

- Détermine l'indépendance d'une classe par rapport au reste du système.
- On parle de "couplage efférent" (noté Ce)
- On compte le nombre de types différents utilisés pour les attributs de la classe, les paramètres des méthodes et les variables locales.
- Plus la valeur est élevée plus la classe :
 - Dépend d'autres classes du projet
 - Est susceptible de devoir être modifiée en cas d'évolutions apportées à d'autres parties du code



Dépendances cycliques de paquetage

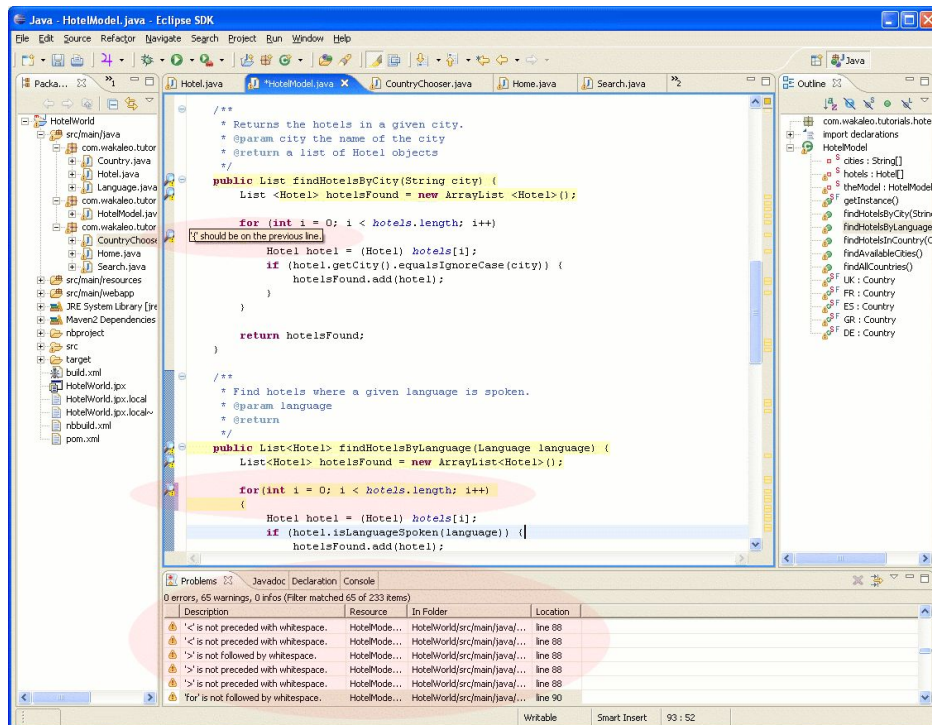
- Détermine la manière dont les classes d'un paquetage référencent des classes extérieures à celui-ci
 - Un paquetage A référence un paquetage B qui référence un paquetage C qui référence à son tour A
 - Les cycles de vie sont alors liés et tous ces éléments ne peuvent ni être utilisés ni être versionnés séparément
 - La modularité du logiciel est cassée



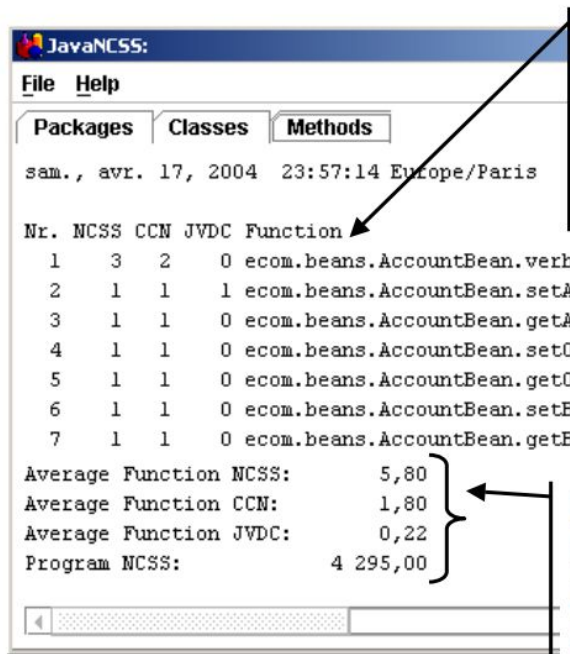
Outils

- De nombreux outils disponibles & complémentaires
 - Java
 - PMD, Checkstyle, JavaNCSS et Findbugs
 - Python
 - autopep8, pylint, ruff
- Stratégie générale
 - Produire des rapports de qualimétrie avec de tels outils (intégration dans les logiciels de développement - plugins)
 - Étudier les rapports produits et les suggestions de corrections
 - Apporter les corrections adéquates

Exemple Checkstyle



JavaNCSS



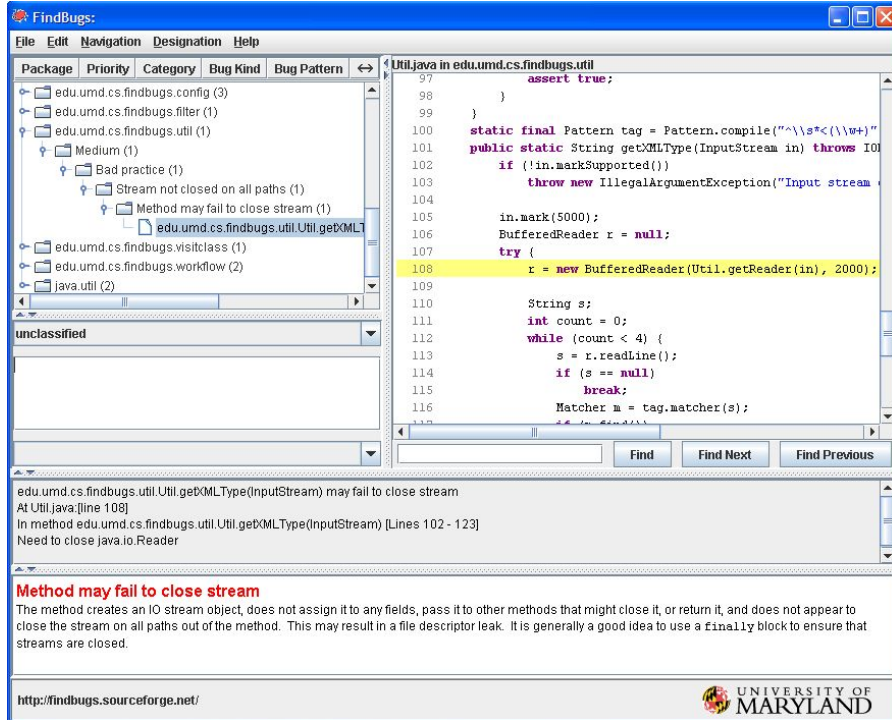
Nr.	NCSS	CCN	JVDC	Function
1	3	2	0	ecom.beans.AccountBean.verbose(String)
2	1	1	1	ecom.beans.AccountBean.setAccountId(int)
3	1	1	0	ecom.beans.AccountBean.getAccountId()
4	1	1	0	ecom.beans.AccountBean.setOwner(String)
5	1	1	0	ecom.beans.AccountBean.getOwner()
6	1	1	0	ecom.beans.AccountBean.setBalance(double)
7	1	1	0	ecom.beans.AccountBean.getBalance()
Average Function NCSS:				5,80
Average Function CCN:				1,80
Average Function JVDC:				0,22
Program NCSS:				4 295,00

Nombre d'instructions sans commentaire, la complexité cyclomatique (CCN), le nombre de commentaires JavaDocs (JVDC) par méthode.

Moyennes par méthode :
d'instructions sans commentaire,
la complexité cyclomatique (CCN),
le nombre de commentaires
JavaDocs (JVDC).



Findbugs



The screenshot shows the FindBugs IDE interface. The left pane displays a project tree with the following structure:

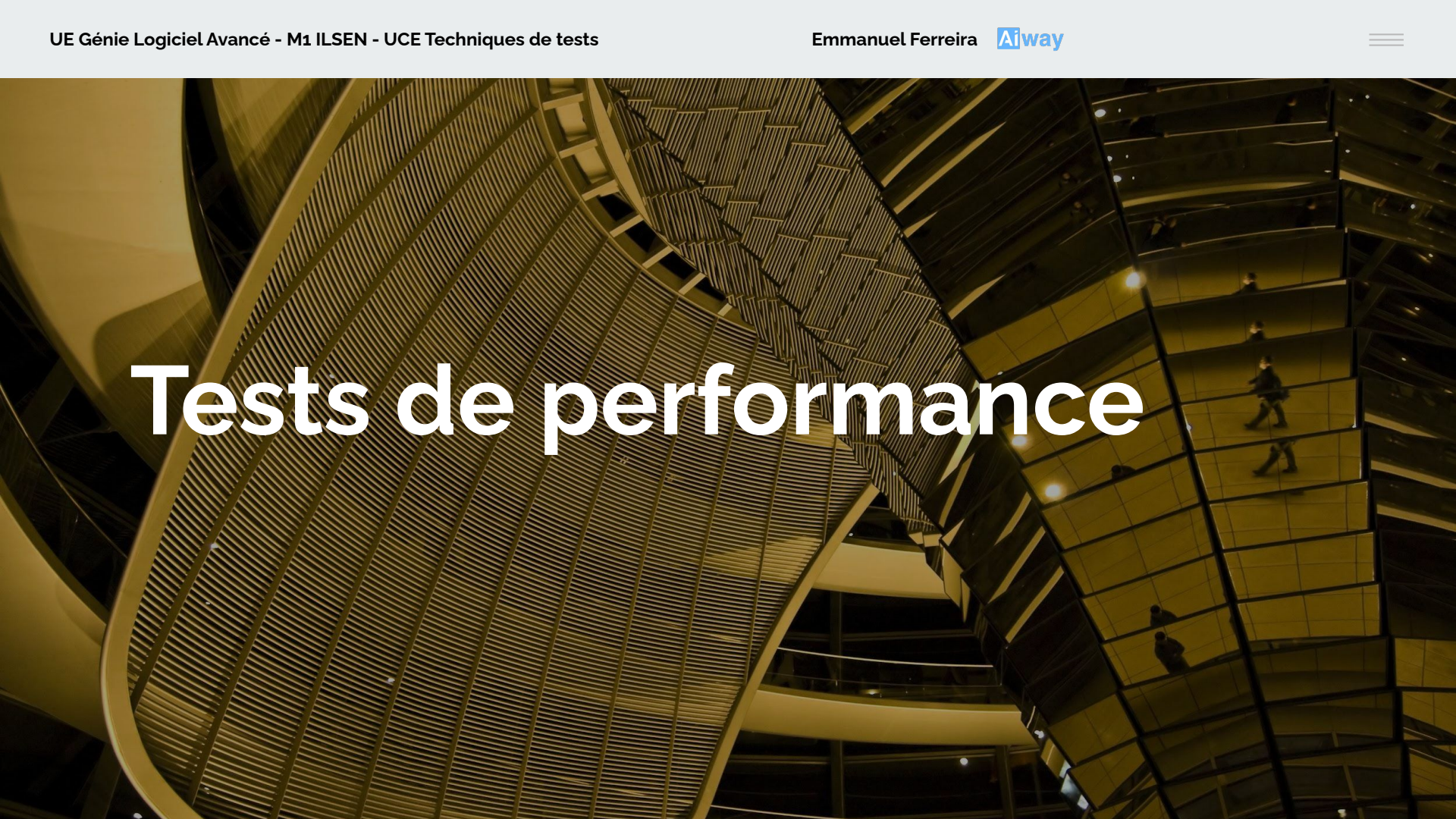
- edu.umd.cs.findbugs.config (3)
- edu.umd.cs.findbugs.filter (1)
- edu.umd.cs.findbugs.util (1)
 - Medium (1)
 - Bad practice (1)
 - Stream not closed on all paths (1)
 - Method may fail to close stream (1)
 - edu.umd.cs.findbugs.util.Util.getBXMLType**
- edu.umd.cs.findbugs.visitclass (1)
- edu.umd.cs.findbugs.workflow (2)
- java.util (2)

The right pane shows the source code of `Util.java` in `edu.umd.cs.findbugs.util`. The code includes an `assert true;` statement, a `Pattern` tag, and a `getBXMLType` method that uses `InputStream` and `BufferedReader`. Line 108, `r = new BufferedReader(Util.getReader(in), 2000);`, is highlighted in yellow.

The bottom pane displays a list of detected bugs:

- unclassified**
- edu.umd.cs.findbugs.util.Util.getBXMLType(InputStream) may fail to close stream**
At Util.java [line 108]
In method edu.umd.cs.findbugs.util.Util.getBXMLType(InputStream) [Lines 102 - 123]
Need to close java.io Reader
- Method may fail to close stream**
The method creates an IO stream object, does not assign it to any fields, pass it to other methods that might close it, or return it, and does not appear to close the stream on all paths out of the method. This may result in a file descriptor leak. It is generally a good idea to use a `finally` block to ensure that streams are closed.

The bottom status bar shows the URL <http://findbugs.sourceforge.net/> and the University of Maryland logo.



Tests de performance



Tests de performance : définition

- Permettent de mesurer la vitesse, les temps de réponse, la stabilité et la fiabilité d'un système lorsqu'il est soumis à une charge de travail donnée.
- Permettent de déterminer l'adéquation entre
 - Les exigences de performance du système
 - La conception et la réalisation du logiciel
 - Les ressources de l'environnement d'exécution
- Des résultats selon trois vues
 - Vue utilisateur
 - Temps de réponse, erreurs
 - Vue application
 - Nombre de transactions, clients connectés
 - Vue technique externe
 - Consommations ressources matérielles et système d'exploitation



Objectifs multiples

- Connaître la capacité du système et ses limites,
- Détecter et surveiller ses points faibles,
- Optimiser ses coûts en infrastructure et en exécution,
- Assurer qu'il fonctionne sans erreurs sous certaines conditions de charge,
- Optimiser les temps de réponse pour améliorer l'expérience utilisateur (UX),
- Reproduire un problème de production,
- Anticiper une montée en charge future, l'ajout d'une fonctionnalité,
- Veiller au bon comportement du système et de ses applications tierces externes, en cas de panne puis de reconnexion, etc.



Tests de performance : quelques déclinaisons

01

- Test de rendement
 - Mesure des temps de réponse d'un système en fonction de sa sollicitation par des utilisateurs dans un contexte réaliste.
 - Rapport entre performances et ressources utilisées (mémoire, bande passante)
- Test de charge (load testing) et de montée en charge (scalability)
 - Performances d'une application sous des charges de travail normales ou prévues
 - Nombre d'utilisateurs simultanés attendus (population cible)
 - En augmentant sa charge de travail par paliers $\Rightarrow$ on recherche les limites en capacité du système, pour valider la qualité de service avant déploiement.
 - Découvrir les éventuels goulots d'étranglement
 - Répond à la question de **charge maximale supportée**.



Tests de performance : quelques déclinaisons

02

- Tests de stress
 - Performances d'une application lorsqu'elle est confrontée à des situations exceptionnelles (explosion du nombre de traitements parallèles, pannes, débordement mémoire, etc.).
 - Identifier le point de rupture de l'application
- Test d'endurance
 - Performances des applications sous des charges de travail normales sur de longues périodes.
 - Vérifier l'absence de problèmes tels que les fuites de mémoire qui peuvent nuire au temps de réponse de l'application.
- Spike testing
 - Évolution des performances sous des charges qui augmente rapidement et de manière répétée sur une courte période (ex. ouverture de billetterie ou soldes)



Comment collecter les données ?

- Vue technique
 - S'appuie sur des outils de l'OS pour les mesures
 - Enregistrement des données du moniteur de ressources
- Vue application et client
 - Placer des « capteurs » dans le code source (instrumentation)
 - Ou utiliser des outils de profilage « capteurs » à l'exécution
- Lancer le test de performance voulu
 - Plateforme de test et moteur de charge
- Dépouiller et analyser les données recueillies



Tests de performance et profilage

- Tests de performance
 - Niveau du système, sous différents types de charge
 - Garantir que le système respecte ses SLA (Service Level Agreement : Contrats de niveau de service).
- Profilage
 - Nécessaire lorsque les tests de performance révèlent un problème.
 - Aide à identifier les parties du système qui contribuent le plus au problème de performances ⇒ où concentrer les efforts.



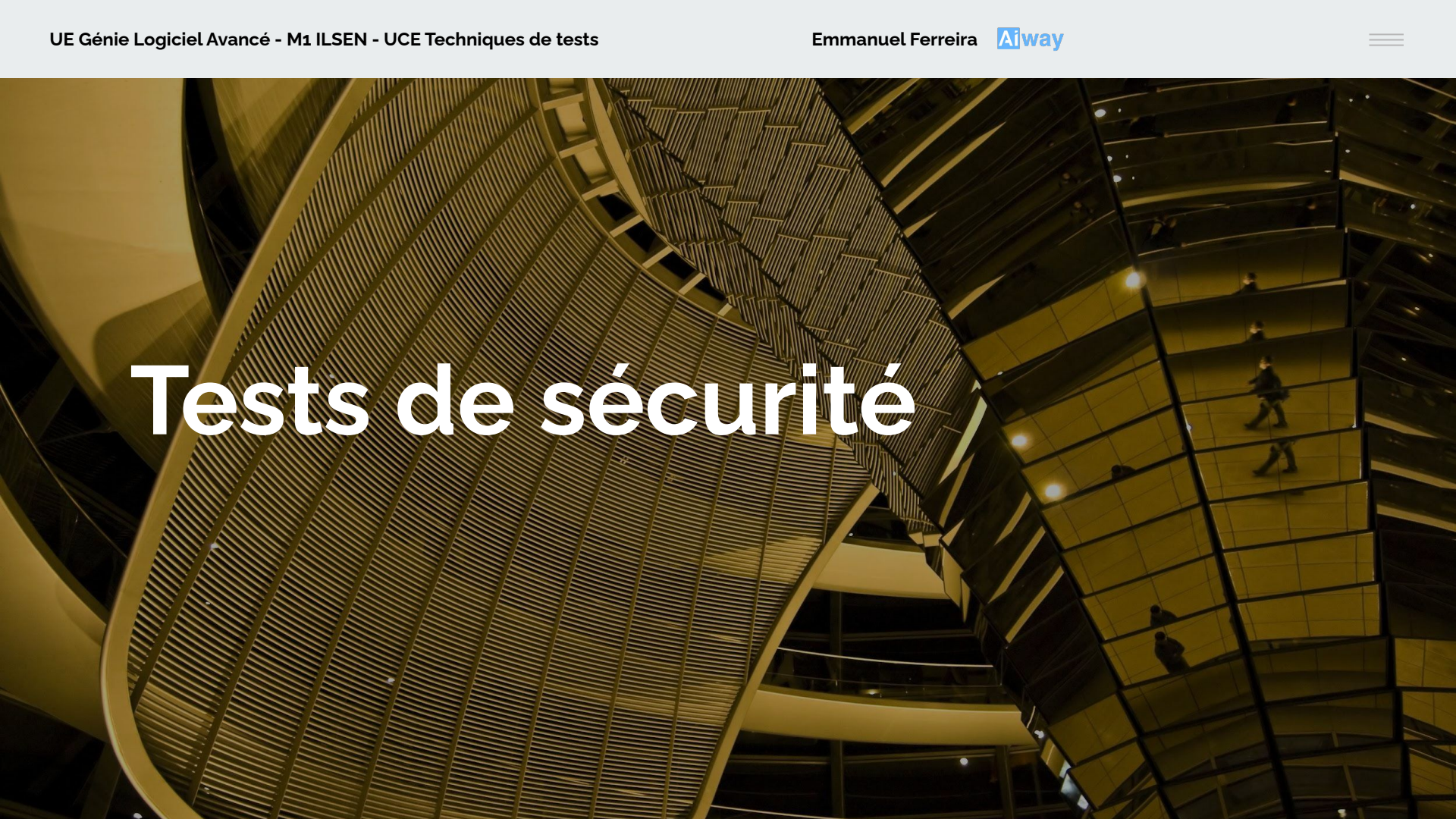
Outils de tests de (montée en) charge

- Répondent à des questions comme :
 - Quelle est la puissance machine nécessaire en fonction du nombre d'utilisateurs/de la volumétrie des traitements ?
 - Est-ce que l'application supporte les accès concurrentiels ?
 - Dead lock, champ statique écrasé, écriture incorrecte dans les fichiers ...
- Objectif
 - Détecter des situations problématiques (non respect des SLA)
- En java :
 - JMeter



Outils de profiling

- Répondent à des questions comme :
 - Quelle partie du code est la plus chronophage ? gourmande en ressources ?
 - Combien de fois est appelée telle méthode ?
 - Y-a-t-il des fuites de mémoire ?
- Objectif
 - Identifier les vecteurs de gains en performance
- En java:
 - JProfiler



Tests de sécurité



Tests de sécurité : objectifs

- Identifier les failles de sécurité : repérer les vulnérabilités exploitables (injections SQL, XSS, erreurs de configuration, mots de passe faibles).
- Protéger la confidentialité des données : vérifier que les données sensibles (données personnelles, bancaires) ne sont pas accessibles sans autorisation.
- Assurer la conformité réglementaire : respecter les exigences légales et normatives (RGPD, PCI-DSS, ISO 27001).



Les 6 domaines de la sécurité applicative

01

- L'Authentification
 - L'identité des utilisateurs est définie (ou vous êtes disposé à accepter des utilisateurs anonymes).
- Autorisation
 - L'accès aux ressources est explicitement autorisé ou refusé aux utilisateurs.
- Non-répudiation
 - Les utilisateurs ne peuvent effectuer une action et ensuite nier l'avoir effectuée.



Les 6 domaines de la sécurité applicative

02

- Confidentialité
 - Les données sont uniquement disponibles pour les personnes qui sont supposées y accéder.
- Intégrité
 - Les données et les ressources du système sont uniquement modifiées de façon appropriée par les personnes habilitées à le faire.
- Disponibilité
 - Les systèmes sont prêts lorsque vous en avez besoins et offrent une acceptabilité.



Insécurité : quelques exemples

01

- Des bases de données MongoDB en libre accès
 - « Des étudiants de l'université de Sarrebruck en Allemagne ont entrepris de répertorier les bases de données MongoDB en libre accès sur le net. Ils en ont trouvé 40 000, dont une appartenant à un important opérateur français. »
- Le réglage par défaut de MongoDB limite en effet les accès de la base de données au seul localhost, mais si l'on supprime la ligne la base est ouverte à toutes les adresses IP.
- Administrateurs négligents et documentations incomplètes



Insécurité : quelques exemples

02

- Plugin Wordpress vulnérable : Slimstat pointé du doigt
 - « Installé sur plus d'un million de sites Wordpress, le plugin de statistiques Slimstat présente une faille le rendant vulnérable aux attaques d'injection SQL »
 - The “secret” was a hashed version of the plugin’s installation timestamp. An attacker could use sites like Internet Archive to approximately guess what year the site was put online (something doable within 10 minutes with most modern CPUs).
- L'attaquant a ensuite la possibilité d'envoyer des requêtes SQL à l'aveugle



Insécurité : quelques exemples

03

- Moonpig : le parfait exemple à ne pas suivre
 - Tokens de connexion = Customer id
 - Requête inconnue = 404 avec toutes les requêtes fonctionnelles de l'API
 - Getcreditcarddetails + identifiant de compte = informations de carte bleue associées au compte
 - Les identifiants sont successifs => automatisation possible de la récupération des informations de carte bleue
 - A exposé les informations personnelles de ses comptes utilisateurs (plus de 3 millions)



Tests de sécurité : en pratique

- Conception de logiciels sécurisés
 - Il est difficile de concevoir un bon logiciel et la sécurité rend la tâche encore plus ardue.
 - Les défauts n'apparaissent pas dans le cadre d'une utilisation normale de l'application.
 - Ce n'est pas toujours la priorité des organisations (cf exemples)
- Deux possibilités de détecter les failles
 - L'analyse du code applicatif (boîte blanche)
 - Tests de sécurité d'application statique (SAST - Static Application Security Testing)
 - Des tests d'attaques malveillantes (automatisés ou non) en
 - Tests de sécurité dynamique des applications (DAST - Dynamic Application Security Testing)
 - Boîte noire : aucune information sur le projet
 - Boîte grise : avec des comptes utilisateurs standards
 - Exemples : les injections SQL, les scripts intersites (XSS), buffer overflow, etc.



Tests de sécurité : des outils

- SAST
 - Find Security Bugs - SpotBugs plugin android et appli web Java
 - Sonarqube
- DAST
 - [sqlmap](#) : automatic sql injection tool
 - Open Web Application Security Project (OWSAP)
 - Communauté travaillant sur la sécurité des applications Web
 - Quelques outils :
 - [OWASP Zed Attack Proxy](#) : web app security scanner
 - OWASP Dependency Check : analyse des dépendances
 - Anchore Engine : analyse de vulnérabilité des containers (JVM, autres)



Test d'intrusion - PenTest : définition

- Simuler une attaque réelle sans perturber la continuité des services de l'entreprise
 - Suit une méthodologie rigoureuse (alignée sur les normes PTES - Penetration Testing Execution Standard - ou OWASP)



Test d'intrusion - PenTest - étape 1/5

- Etape 1 : Cadrage
 - Définition du périmètre (Scope) : identification précise des cibles (adresses IP, applications web, réseaux Wi-Fi, APIs, locaux).
 - Choix du type de test :
 - Boîte noire : l'auditeur n'a aucune information préalable (approche pirate externe)
 - Boîte grise) : l'auditeur possède des accès limités (ex. compte utilisateur standard)
 - Boîte blanche : accès complet au code source, à l'architecture et aux configurations.
 - Signature de l'autorisation d'intrusion (Mandat d'audit) : document légal protégeant l'auditeur et autorisant formellement les tests à des dates précises.



Test d'intrusion - PenTest - 2/5

- Etape 2 : Reconnaissance et collecte d'informations
 - L'objectif est d'amasser un maximum de données sur l'entreprise et l'infrastructure ciblée.
 - Reconnaissance passive (OSINT) : recherche d'informations publiques sur l'entreprise, adresses e-mail de collaborateurs, sous-domaines, ou fuites de données sur le Dark Web.
 - Reconnaissance active : identification des serveurs actifs, découverte des ports ouverts et cartographie du réseau cible..



Test d'intrusion - PenTest - 3/5

- Etape 3 : Cartographie et analyse des vulnérabilités
 - L'auditeur analyse les services identifiés pour repérer les failles potentielles.
 - Identification des services : détection des versions de logiciels, systèmes d'exploitation et frameworks utilisés.
 - Recherche de vulnérabilités : croisement des composants identifiés avec les bases de failles connues (CVE) et balayage à l'aide d'outils automatisés.
- Etape 4 : Exploitation
 - L'auditeur tente d'exploiter les failles découvertes pour s'introduire dans le système.
 - Attaque ciblée : injection de payloads (SQLi, XSS, Buffer Overflow), exploitation d'erreurs de configuration ou attaque par force brute sur les identifiants.
 - Ingénierie sociale (optionnel) : envoi d'e-mails de phishing ciblés pour récupérer des accès auprès des collaborateurs.



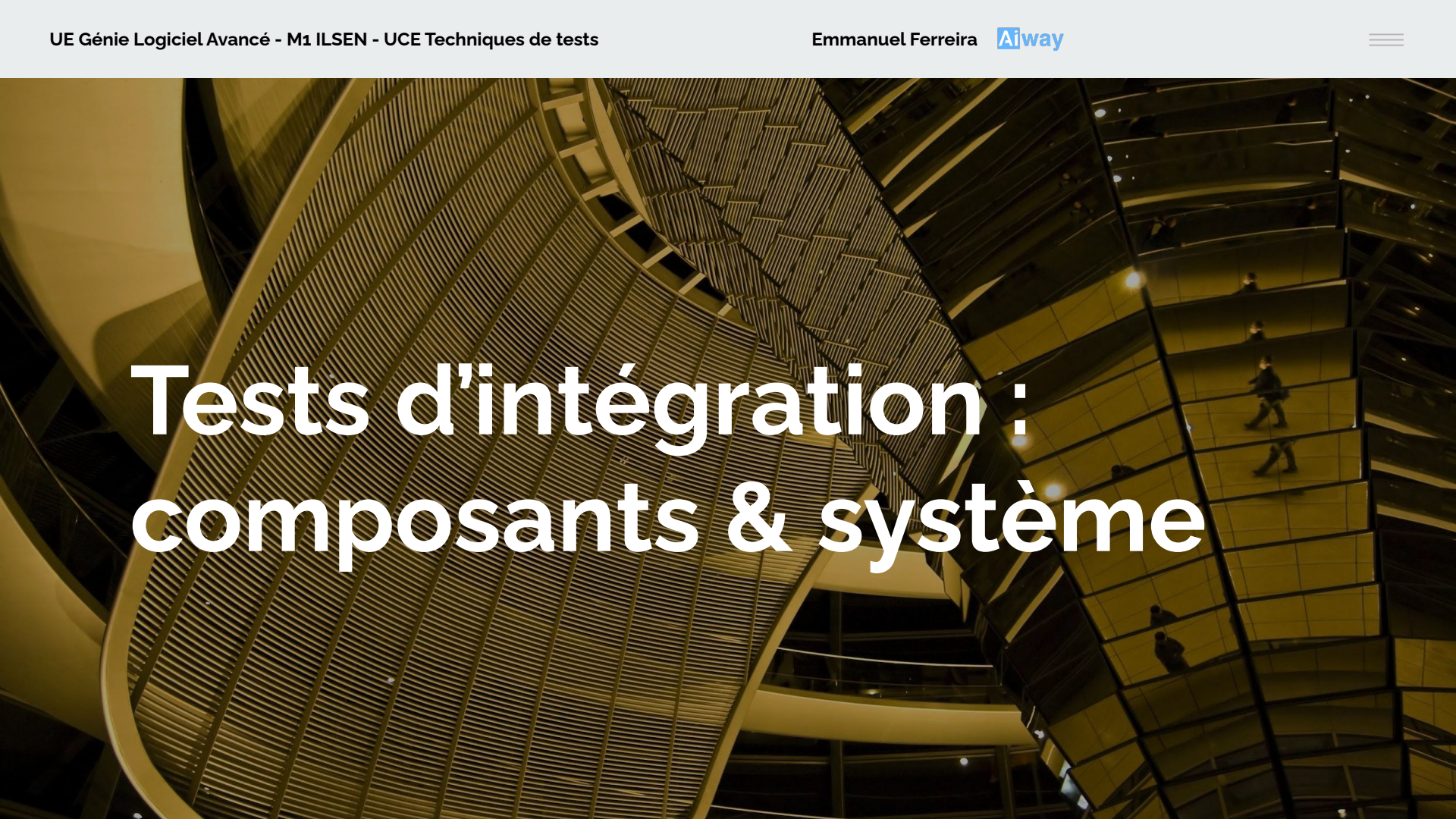
Test d'intrusion - PenTest - 4/5

- Etape 5 : Post-exploitation et élévation de privilèges
 - Une fois le premier accès obtenu, l'auditeur cherche à mesurer la portée réelle de l'attaque.
 - Élévation de privilèges : passage d'un compte utilisateur basique à un accès administrateur (Root / Domain Admin).
 - Mouvement latéral : rebond d'un serveur compromis vers d'autres machines sensibles du réseau interne.
 - Évaluation de l'impact : démonstration de la capacité à accéder à des données hautement confidentielles (sans altérer la production).



Test d'intrusion - PenTest - 5/5

- Étape 6 : Rapport et restitution
 - Le livrable final qui synthétise l'ensemble de la mission pour les équipes techniques et la direction.
 - Synthèse décisionnelle : vue d'ensemble du niveau de risque globale destinée au management.
 - Détail technique : liste chronologique des failles exploitées avec preuve de concept (PoC) et captures d'écran.
 - Plan de remédiation : recommandations concrètes et priorisées par niveau de sévérité (Critique, Élevé, Moyen, Faible) pour corriger les failles.
- Étape 7 : Contre-visite
 - Après la phase de correction par les équipes informatiques, une vérification est effectuée pour confirmer que les vulnérabilités identifiées ont été comblées et qu'aucune nouvelle faille n'a été introduite.



Tests d'intégration : composants & système



Tests d'intégration : définition

- 2 types : composants et système
 - Composants :
 - Vérifier si plusieurs unités de code fonctionnent bien ensemble
 - Environnement de test assez proche du test unitaire, c'est-à-dire de manière isolée, sans lien avec des composants extérieurs et ne permettant pas le démarrage d'une vraie application
 - Boîte blanche/grise, dans la mesure où l'on est amené à regarder les relations entre les composants pour vérifier leur bonne coopération
 - Système :
 - Vérifier le fonctionnement de système complet (ou d'une sous partie) en prenant des scénarios ⇔ cas d'utilisation du logiciel)
 - Boîte noire
 - Ne tient pas compte de l'implémentation ou des composants sous-jacents (focus fonctionnel)
 - Il est fortement recommandé que l'équipe qui réalise ces tests soit distincte de l'équipe de développement



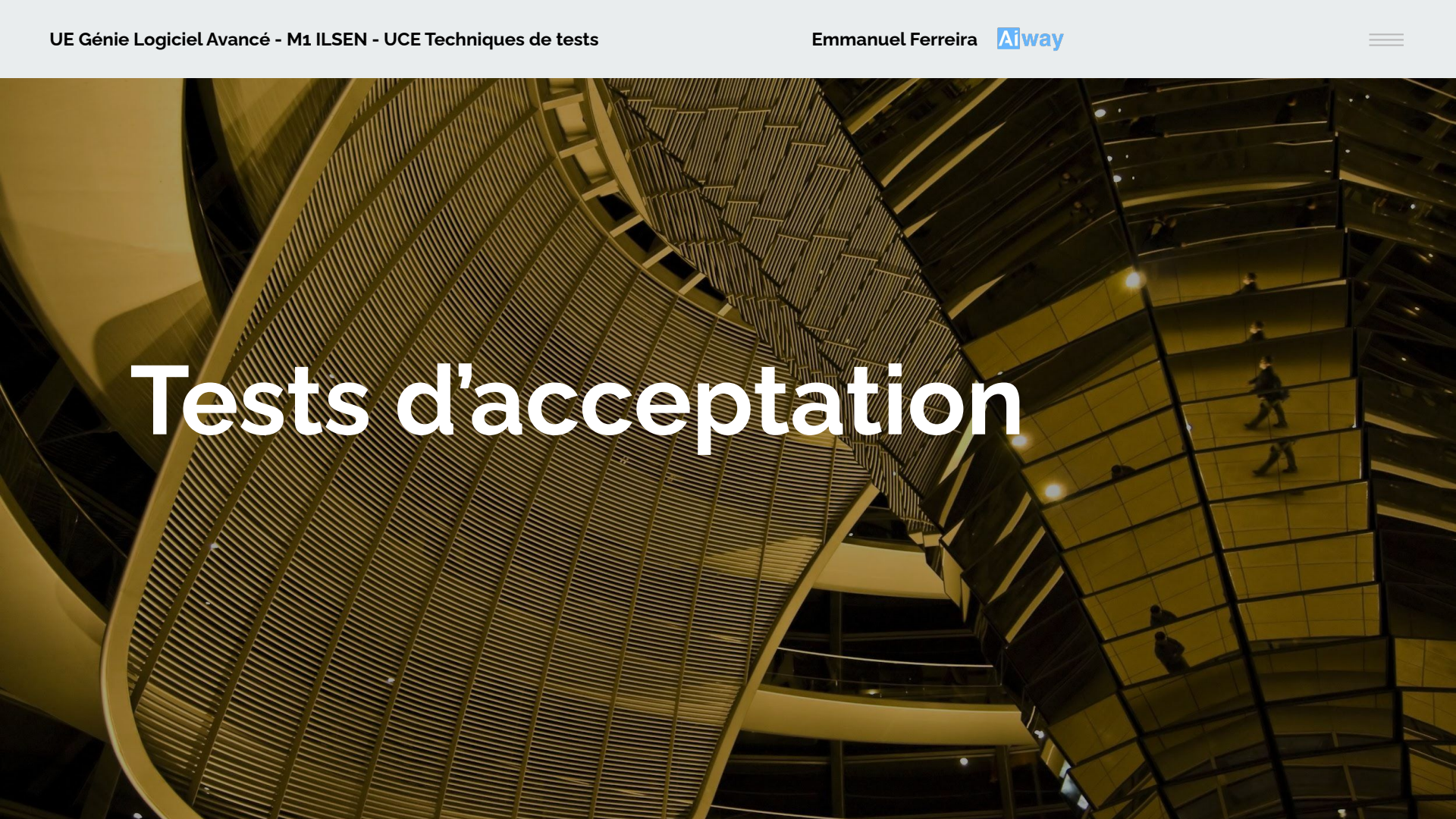
Tests d'intégration

- Test d'un ensemble d'unités qui coopèrent;
 - But : détecter des erreurs dans leur interopérabilité, la mauvaise utilisation d'une interface
- Interconnexion de composants (niveau macro); commence très tôt en objet (niveau micro): une classe est typiquement composée d'objets d'autres classes;
- Bien repérer l'inter-dépendance des classes pour choisir un ordre d'intégration :
 - Si les dépendances forment un arbre (un ordre partiel), alors on peut intégrer simplement de bas en haut;
 - S'il y a un cycle de causalité (A dépend de B qui dépend de A), fréquent :
 - On émule une des classes (par ex A);
 - On teste B avec l'émulation de A;
 - On teste A avec B;
 - On reteste B avec le vrai A



Test système

- Objectif : vérifier que le système End-to-End répond aux exigences fonctionnelles et non fonctionnelles
 - Fonctionnel : on teste les scénarios intéressants déterminés lors de l'analyse (use cases, sequence diagrams) - parcours client
 - Boîte noire uniquement : la spécification est alors le seul critère de référence.
 - Non fonctionnel : on teste les performances
- L'application à tester est complètement intégrée :
 - Dans un environnement ISO-production complet (bases de données, réseaux, dépendances) - y compris avec les autres applications qui seront utilisées conjointement
- Equipe QA / testeurs dédiés



Tests d'acceptation



Tests d'acceptation

- Testent le système afin de s'assurer qu'il est conforme aux attentes du client
 - Exigences
- Menés avec des utilisateurs pilotes
 - Client peut ainsi valider tout ou partie d'une fonctionnalité d'un logiciel
- Par nature ce sont des tests fonctionnels
- Conseil :
 - Associer très tôt les utilisateurs à une première portion du logiciel permettra de mieux connaître les critères d'acceptation par les utilisateurs, et, au besoin, d'ajuster certains choix (ergonomie, etc.)



Exigence

- **Forme d'expression de besoins.**
 - Énoncé décrivant un besoin que doit satisfaire le système à réaliser.
 - Une exigence ne se limite pas à la description d'une fonctionnalité, elle peut aussi établir des critères de disponibilité, de performance, de sécurité...
 - Système est présenté sous la forme d'une liste d'exigences (souvent plusieurs centaines).
- **Caractéristiques d'une exigence**
 - Elle est non ambiguë. Elle ne doit donc pas laisser de place à l'interprétation.
 - Elle est délimitée, c'est-à-dire que son intention doit être suffisamment précise.
 - éléments chiffrés : le temps de réponse de l'application doit être inférieur à X seconde
 - Elle doit être testable.
 - Une seule exigence ne suffit généralement pas à exprimer le détail d'une fonctionnalité.



Tests d'acceptation : en pratique

- S'écrivent dans un langage proche du client
 - FitNesse <http://fitnesse.org/>
 - Concordion <http://www.concordion.org/>
 - Gherkin
- Pour autant, pas tous nécessairement à la portée des clients
- Doivent néanmoins pouvoir servir de support de discussion pour spécifier les fonctionnalités.



Agilité et tests d'acceptation

- Méthodes agiles utilisent des cycles de développement courts pendant lesquels sont pris en charge la réalisation de "User stories" (fonctionnalités) ⇒ 2 approches pour garantir l'adéquation de ses User Stories avec les besoins métiers :
 - ATDD – Acceptance Test-Driven Development
 - Approche : visuelle et collaborative
 - Principe : s'appuie sur des représentations graphiques du workflow de test (schémas, diagrammes) pour échanger, affiner et valider les scénarios au niveau de la fonctionnalité ou du processus métier à implémenter.
 - BDD - Behavior-Driven Development
 - Approche : basée sur le comportement.
 - Principe : utilise un langage naturel structuré (le langage Gherkin) pour rédiger les scénarios de test. Ces scénarios définissent directement les critères d'acceptation de chaque User Story sous la forme Étant donné / Quand / Alors.



Behavior Drive Development

- Créer des tests fonctionnels avec un langage naturel compris de tous (Gherkin).
 - Les développeurs sont guidés par les comportements attendus par les utilisateurs
- Voici un exemple simple dont les parties en gras représentent la structure de base obligatoire pour écrire ces tests :
 - **Etant donné que** je suis le client sur la fiche produit
 - **Quand** je clique sur ajouter au panier sur le produit d'identifiant "255"
 - **Et que** le stock est à "3"
 - **Alors** le produit se rajoute au panier.
- Cette phrase sera automatiquement transformée par des applications en ligne de code ; les développeurs n'auront plus qu'à remplir les comportements au sein de l'enveloppe (des fonctions en code) pour écrire le test.
 - Quand le test est bien écrit, les développeurs écriront les fonctions résultantes.



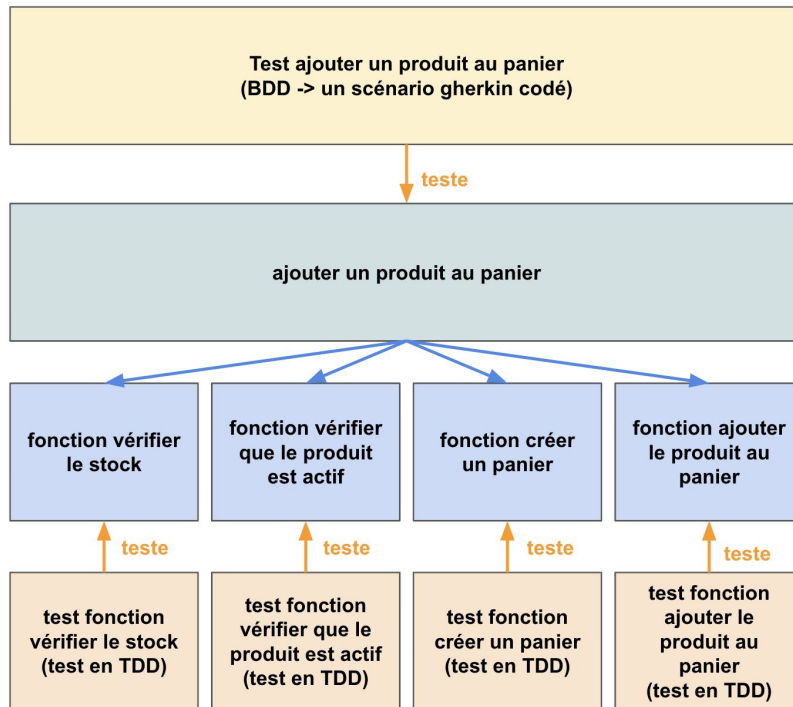
TDD et BDD des approches complémentaires

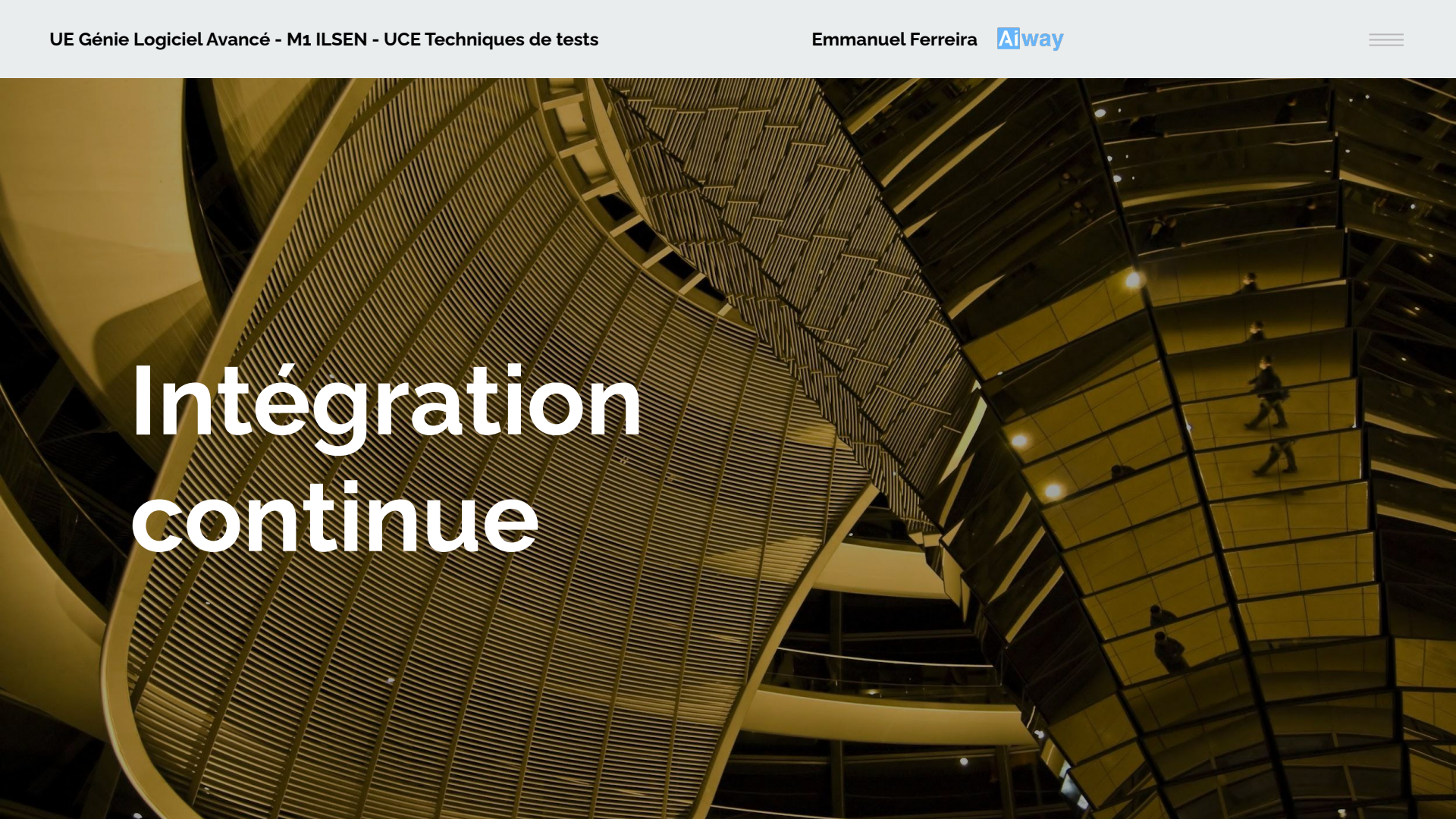
01

- TDD guide les développements, fonction par fonction (feature)
- BDD permet de tester l'ensemble des comportements attendus par la user-story (scénario) à développer.
 - En pratique scénario $\Leftrightarrow$ plusieurs fonctions

TDD et BDD des approches complémentaires

02





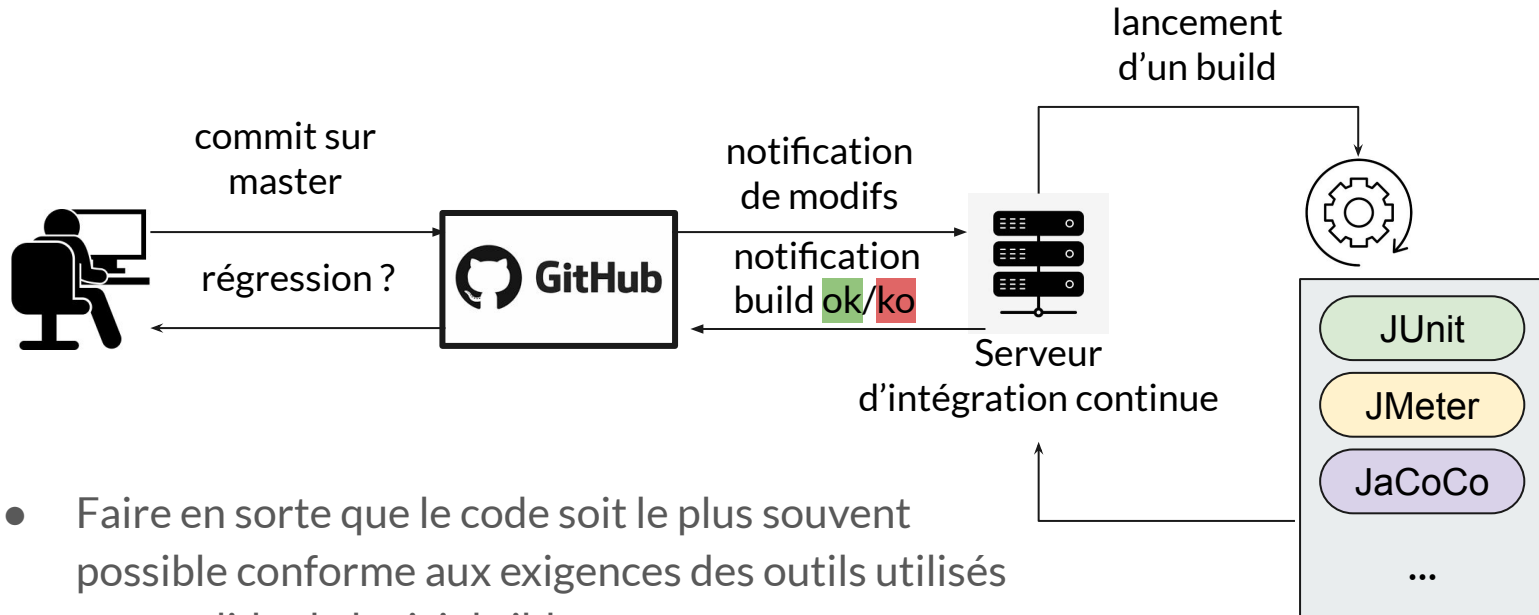
Intégration continue



Intégration continue : définition

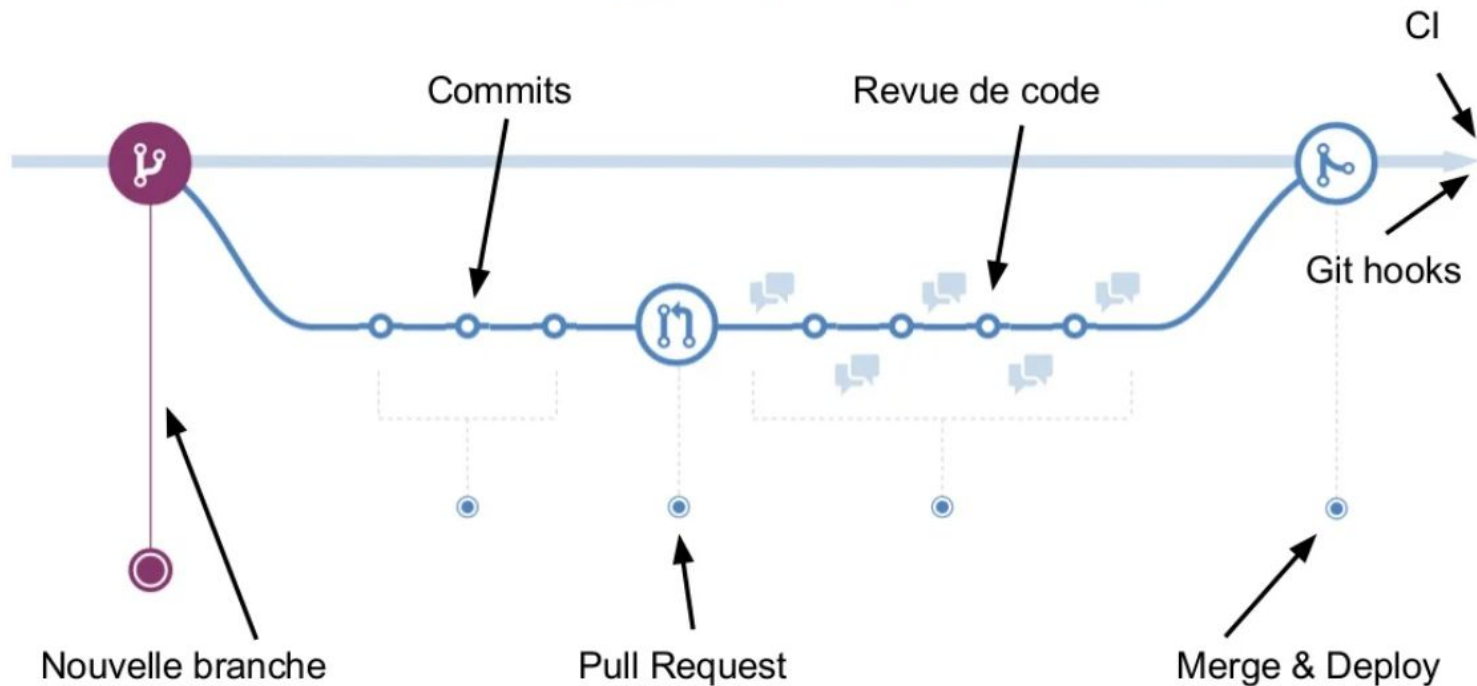
- Solution pour qu'une série de tests (unitaires, intégrations, performance, etc.) - soient lancés de façon systématique (**build**) en guise de validation d'un événement important
 - Fusion d'une branche git sur laquelle vous avez travaillé dans la branche master
 - Déploiement en (pré)production.
 - etc.
- Objectifs :
 - Vérifier que les modifications apportées ne produisent pas de régression
 - Absence de problèmes d'intégration
 - Automatisation des suites de tests existantes

Intégration continue : en image



- Faire en sorte que le code soit le plus souvent possible conforme aux exigences des outils utilisés pour valider le logiciel cible

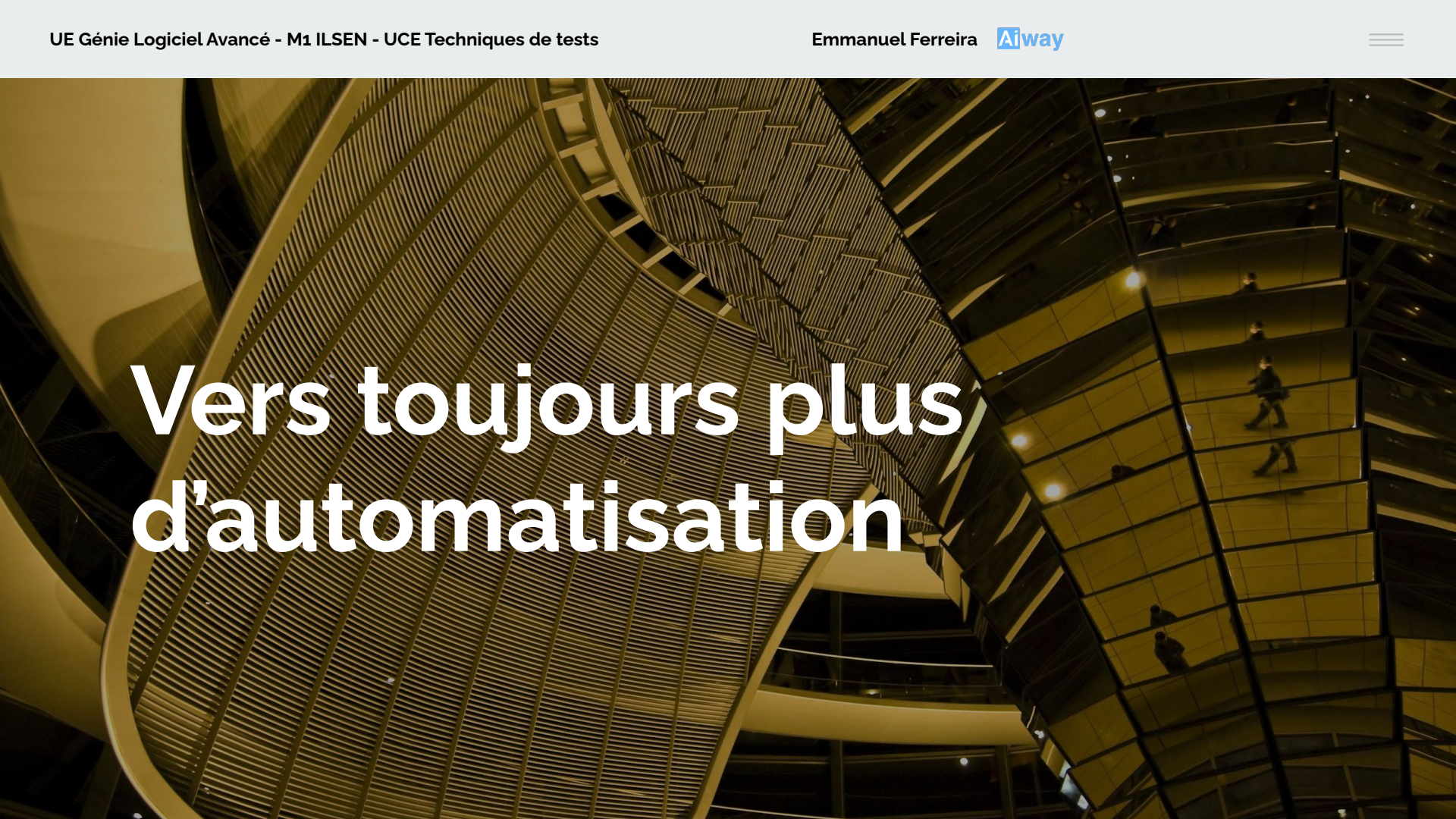
Workflow GIT typique





Intégration continue : à l'heure du choix

- En fonction de ce qui est nécessaire de valider et les intégrations possibles avec l'outil d'intégration continue, il est possible d'ajouter de nombreux autres outils de validation.
- Quelques outils
 - Circle CI
 - Gitlab CI
 - Jenkins
 - Travis CI
 - ...
- Comment choisir le bon outil d'intégration continue ?
 - Pour une entreprise : Coût / popularité / pérennité
 - Pour vous : il est inutile de chercher à vous former sur tous les outils ! Le tout est d'en comprendre le fonctionnement général et de vous référer aux documentations le moment venu



Vers toujours plus d'automatisation



Le présent et l'avenir des tests

- Nécessité de tester un logiciel à chaque phase de son développement ⇒ reconnue par les professionnels de l'IT
 - Cependant cela reste une activité coûteuse...
- Face à ce constat ont été développés des outils et des techniques qui permettent d'automatiser une partie des tests logiciels.
 - Intégration continue
- Cette automatisation n'est pas une solution miracle
 - Il faut encore écrire les scripts, les maintenir à jour et les faire évoluer dans le temps (pesticide) !
- Vers toujours plus d'automatisation
 - Place de l'IA dans ce processus ?



L'IA et les tests

01

- Détection automatique d'erreurs
 - Estimer où le risque de défauts est le plus élevé (à partir des expériences passées).
 - Priorisation des composants à tester ⇒ optimiser l'effort de test
- Détection de vulnérabilités
 - Déterminer automatiquement les différents chemins que les utilisateurs finaux peuvent emprunter pour signaler les failles pour la protection du code/des données sensibles
- Reconnaissance de forme
 - Les intelligences artificielles relatives à la vision par ordinateur peuvent permettre de reconnaître des éléments de l'interface graphique par leurs caractéristiques.
 - Automatiser une partie des tests d'affichage :
 - Détecter qu'un élément visuel est bien affichée
 - Identification d'un nombre des changements trop importants d'une version à l'autre



L'IA et les tests

02

- Suivi automatique de la couverture des tests :
 - Détection automatique des exigences de tests présents et/ou modifiés grâce à l'analyse continue des changements apportés à l'application et au traitement en langage naturel (NLP) des besoins définis
- Génération automatique
 - Estimation automatique du jeu de test le plus pertinent compte tenu de l'état courant du code
 - Élaboration de scénarios de tests pertinents au regard des exigences
- Autoguérison :
 - Correction en continu et en automatique des problèmes de test défectueux.



L'IA et les tests

03

- Aujourd'hui les outils qui intègrent de l'IA dans le test se concentrent sur les tests unitaires, intégration, affichage et générations de données de test et se positionnent comme outils d'aide aux testeurs
- Demain, aidés de techniques IA de plus en plus sophistiquées il y a de grande chance qu'ils puissent prendre en charge les tests de plus haut-niveau (intégration, système) voire de pré-acceptation $\Rightarrow$ modèle utilisateur (modéliser le comportement/réaction des utilisateurs, etc.)



Merci pour votre attention !

